

第 39 章: Decorators (装饰器)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 这是《Learning Python》面向高级读者的核心章节之一, 也是继第 38 章 (Managed Attributes, 管理属性) 之后的又一座技术高峰。全书此前散落各处的装饰器知识在这里汇流: 我们在第 29 章 (抽象超类)、第 32 章 (类的杂项)、第 38 章 (property 内建) 中都已零散见过 @ 语法, 本章把它们收拢成统一的地图——装饰器归根结底就是"名字重绑定 (name rebinding)"的一次语法糖。本章同时是两个大型实战案例 (属性隐私 + 参数范围校验) 的舞台, 也是通往第 40 章 (metaclasses, 元类) 的桥头堡。学完本章, 你将具备"用代码管理代码"的能力——这是区分"会用 Python"与"给 Python 造工具"的分水岭。

39.1 开篇引言

英文原文

In Chapter 32's survey of class odds and ends, we met properties and static and class methods, took a quick look at the @ decorator syntax Python offers for declaring them, and previewed decorator coding techniques. We also met function decorators briefly while e

exploring the property built-in in Chapter 38, in the context of abstract superclasses in Chapter 29, and in capsule form in Chapter 19. This chapter picks up where all this previous decorator coverage left off. Here, we'll dig deeper into the mechanics of decorators and study more ways to code new decorators ourselves with tools like arguments and nesting. As we'll find, other concepts we studied earlier—especially state retention—show up regularly in decorators. This is a somewhat advanced topic, and decorator construction tends to be of more interest to tool builders than to application programmers. Still, given that decorators are becoming increasingly common in popular Python frameworks, a basic understanding can help demystify their role, even if you're just a decorator user. Besides covering decorator construction details, this chapter serves as a more realistic case study of Python in action. Because its examples grow larger than many of the others we've seen in this book, they better illustrate how code comes together into more complete systems and tools. As an extra perk, some of the code we'll write here may be used as general-purpose tools in your day-to-day programs.

中文翻译

在第 32 章对类"杂项知识"的考察中，我们接触了 property、静态方法（static method）和类方法（class method），初步浏览了 Python 为声明它们而提供的 @ 装饰器语

法，并对装饰器的编码技巧做了预热。在第 38 章探索 property 内建函数时、在第 29 章讨论抽象超类的语境中、以及在第 19 章的浓缩形式里，我们也都会过函数装饰器的面。本章正是从前面这些装饰器知识停下的地方继续深入。在这里，我们将深入研究装饰器的运作机制，并借助参数 (arguments) 与嵌套 (nesting) 等工具，学习更多自己编写新装饰器的方法。我们会发现，之前学过的其他概念——尤其是状态保存 (state retention) ——会在装饰器中反复登场。这是一个稍微高级的话题，而装饰器的构造往往对 "工具构建者 (tool builders)" 比对 "应用程序员 (application programmers)" 更有吸引力。不过，由于装饰器在流行的 Python 框架中已经越来越常见，即使你只是一个装饰器的使用者，对它的角色有个基本理解，也能帮助你去除神秘感。除了涵盖装饰器的构建细节之外，本章还充当一个更贴近现实的 "Python 实战案例研究"。因为本章的示例比本书中许多其他示例都要大，它们能更好地演示代码是如何拼装成更完整的系统与工具的。作为额外福利，我们这里写出的部分代码可以直接用作你日常程序中的通用工具。

深度理解

·核心概念：装饰器 (decorator) 是一种 "在函数/类定义语句收尾的瞬间自动运行一段代码" 的机制。它不是某个只能在函数前面加一个 @ 的小记号，而是一整套 "由 callable 管理 callable" 的协议。本章是散落在前文各处 @ 用法的总中枢。

·底层实现：装饰器的本质是名字重绑定 (name rebinding) ——@dec 写在 def f 之前，等价于函数定义结束后

自动执行 `f = dec(f)`。理解了这一条，装饰器机制就已经理解了一半。

·设计原因：Lutz 点破了它的现实意义——装饰器正在流行的 Python 框架中普及（如 Flask 的 `@app.route`、Django 的 `@loginrequired`、PyTorch 的 `@autocast`）。框架的使用者即使不会自己写装饰器，也必须看得懂它们。

·解决什么实际问题：把“横切关注点”（日志、计时、校验、注册、权限）从业务代码中剥离出来，做到一次声明、处处生效、显式可见。这些逻辑若不用装饰器，就得散落在每个调用点，极易遗漏。

·初学者误区：以为装饰器是“魔法语法”。其实它毫不新奇——它只是把 `f = dec(f)` 写成 `@dec` 的等价形式。真正的难点不在于 `@` 本身，而在于如何编写那个要返回 callable 的 `dec`。

39.2 What's a Decorator? (什么是装饰器?)

英文原文

Simply put, decoration is a way to specify management or augmentation code for functions and classes. Decorators themselves take the form of callable objects (e.g., functions) that process other callable objects. As suggested earlier in this book, Python decorators

come in two related flavors: Function decorators, added first, do name rebinding at function definition time, providing a layer of logic that can manage functions and methods or later calls to them. Class decorators, added later, do name rebinding at class definition time, providing a layer of logic that can manage classes or the instances created by later calls to them. In short, decorators provide a way to insert automatically run code at the close of function and class definition statements—at the end of a `def` for function decorators and at the end of a `class` for class decorators. Such code can play a variety of roles, as described in the following sections.

中文翻译

简单来说，装饰（decoration）是一种为函数和类指定“管理代码或增强代码”的方式。装饰器本身采用“可调用对象（callable objects，例如函数）”的形式，去处理其他可调用对象。正如本书前文所暗示的，Python 装饰器分两个相关门类：函数装饰器（function decorators），最先加入，在函数定义时执行名字重绑定，提供一层逻辑去管理函数与方法（methods），或管理对它们的后续调用。类装饰器（class decorators），后来加入，在类定义时执行名字重绑定，提供一层逻辑去管理类，或管理由后续对其的调用所创建的实例。总之，装饰器提供了一种方式，能在函数和类定义语句收尾时插入自动运行的代码——函数装饰器插在 `def` 语句的末尾，类装饰器插在 `class` 语句的末尾。这样的代码可以扮演多种角色，正如以下各节所述。

深度理解

- 核心概念：装饰器提供的钩子是"定义语句结束时自动运行"。这与普通赋值语句一样平凡：`def` 与 `class` 语句本来就是在运行时执行的，装饰器只不过是这一语句末尾追加了一次重赋值。
- 底层实现：CPython 编译 `@dec` 时，会把它编译成"定义函数/类 → 调用 `dec` → 把结果赋回原名字"的字节码序列。装饰器在模块被 `import` 时执行一次，而不是在每次调用时执行。
- 设计原因：函数先加入，社区先在函数上趟平了路，后来才把同样的模型推广到类。两类装饰器共享同一套语法，降低了学习迁移成本。
- 解决什么实际问题：让"给函数加日志、缓存、鉴权"这类横切逻辑从每个调用点收拢到一处静态声明；类装饰器则把"每次创建实例都要手动包一层代理（proxy）"的苦役压缩成一条 `@` 行。
- 初学者误区：以为"装饰器装饰的是函数本身"。准确的说法是——装饰器的返回值（可能还是原函数，也可能是别的任意可调用对象）被绑定到原名字上。

代码分析

本章开篇是概念介绍，其"代码"其实是一幅映射图：

```

#
@decorator          #
def F(arg): ...
F(99)               #

#
def F(arg): ...
F = decorator(F)    #
F(99)               # decorator(F)(99)

```

Python 解释器看到 `@decorator` 时，会先编译 `def F` 创建函数对象 `F`，紧接着立刻把 `F` 作为参数调入 `decorator`，最后把返回结果重新绑定到 `F` 这个名字上。因此之后 `F(99)` 不再调用原来的函数体，而是调用 `decorator(F)` 的返回值——可能是原始函数，也可能是一个包装器（wrapper）。设计思想：把“事后追加行为”变成定义时刻的确定性动作，让增强逻辑与函数本体解耦。

39.3 Managing Calls and Instances（管理调用与实例）

英文原文

In typical use, this automatically run code may be used to augment calls to functions and classes. It arranges this by installing wrapper (a.k.a. proxy) objects to be invoked later: Call proxies: Function decorators install wrapper objects to intercept later function calls and process them as needed, usually passing the call on to the original function to run the managed action.

Interface proxies: Class decorators install wrapper objects to intercept later instance-creation calls and process them as required, usually passing the call on to the original class to create a managed instance. Decorators achieve these effects by automatically rebinding function and class names to other callables at the end of def and class statements. When later invoked, these callables can perform tasks such as tracing and timing function calls, managing access to class instance attributes, and so on.

中文翻译

在典型使用中，这段自动运行的代码可以用来增强对函数和类的调用。它的实现方式是：安装包装器（wrapper）（又名代理（proxy））对象，留待之后被调用：调用代理（call proxies）：函数装饰器安装包装器对象，以拦截后续的函数调用并按需处理——通常把调用转发给原函数，去执行被管理的动作。接口代理（interface proxies）：类装饰器安装包装器对象，以拦截后续的实例创建调用（instance-creation calls）并按需处理——通常把调用转发给原类，从而创建被管理的实例。装饰器通过在 def 和 class 语句的末尾自动把函数名、类名重绑定到其他可调用对象，来实现这些效果。当这些可调用对象后来被调用时，就可以执行诸如跟踪（tracing）与计时（timing）函数调用、管理类实例属性的访问等任务。

深度理解

·核心概念：“调用代理”与“接口代理”是把所有装饰器角色

一分为二的两个原型。函数装饰器拦截"对函数的调用"; 类装饰器拦截"对类的实例化调用"; 而拦截实例的接口访问, 靠的是包装器实例上的 `getattr`。

·底层实现: 安装代理的本质, 是把名字从"原来的可调用对象"改成"另一个可调用对象 (wrapper)"。调用发生时, 解释器只认识新的代理对象, 原对象被埋藏在代理内部。

·设计原因: "拦截 → 处理 → 放行给原对象"是代理模式的普适优点; 装饰器把"安装代理"这个动作自动化了, 让使用者无需手工改写调用点。

·解决什么实际问题: 想给所有调用加密、鉴权、缓存、记录、计时, 又不想去每个调用点改写——代理是标准解法, 装饰器让安装代理只需要一行 `@`。

·初学者误区: 以为包装器必须返回"函数"。实际上包装器只须是"可调用"即可; 返回 `lambda`、类实例 (带 `call`) 都完全可以。

39.4 Managing Functions and Classes (管理函数与类)

英文原文

Although most examples in this chapter deal with using wrappers to intercept later calls to functions and classes, this is not the only way decorators can be used: Function managers: Function decorators can also b

used to manage function objects instead of or in addition to later calls to them—to register a function to an API, for instance. Our primary focus here, though, will be on their more commonly used call-wrapper application. Class managers: Class decorators can also be used to manage class objects directly, instead of or in addition to instance-creation calls—to augment a class with new methods or data, for example. Because this role intersects strongly with that of metaclasses, we'll explore additional decorator use cases in the next chapter. As detailed there, both tools run at the end of the class creation process, but class decorators often offer a lighter-weight solution. In other words, function decorators can be used to manage both function calls and function objects, and class decorators can be used to manage both class instances and classes themselves. By returning the decorated object itself instead of a wrapper, decorators become a simple post-creation step for functions and classes. Regardless of the role they play, decorators provide a convenient and explicit way to code tools useful both during program development and in live production systems.

中文翻译

虽然本章的大多数示例都是利用包装器去拦截对函数和类的后续调用，但这并不是装饰器唯一的使用方式：函数管理器（function managers）：函数装饰器也可以用来管理函数

对象本身，而不是（或不限于）对它们的后续调用——例如把函数注册到一个 API。不过我们这里的重点，仍然是装饰器更常见的"调用包装器"应用。类管理器（class managers）：类装饰器也可以直接管理类对象本身，而不是（或不仅是）处理实例创建调用——例如为类扩充新方法或新数据。因为这个角色与元类（metaclasses）的角色高度重叠，我们将在下一章中探讨更多的装饰器用例。正如那里将详述的，两种工具都在类创建过程的末尾运行，但类装饰器通常提供更轻量级的解决方案。换言之，函数装饰器既能管理函数调用，也能管理函数对象；类装饰器既能管理类实例，也能管理类本身。如果装饰器不加包装器、而是直接返回被装饰对象本身，那么它就为函数和类提供了简单的"创建后处理步骤（post-creation step）"。无论扮演哪种角色，装饰器都提供了一种便捷而显式的方式去编写工具——这些工具在程序开发期和真实生产系统中都大有用途。

深度理解

- 核心概念：装饰器的能力光谱有两大极端——管理"原函数/原类本身"（管理器）vs. 管理"对它们的后续调用"（包装器）。
- 底层行为：当作管理器时，装饰器 dec 拿到 O，处理完 O 后 return O（返回原对象）；当作包装器时，dec 返回别的可调用对象。
- 设计原因：注册到 API、初始化属性这类"一次性"需求不需要代理，只需要"创建结束时顺手跑一段代码"——这正好是装饰器的定位。
- 解决什么实际问题：例如把一组函数注册进路由表、给函

数附加元数据、把类登记到框架的清单中。若没有装饰器，你得在定义之后到处显式调用注册函数。

·初学者误区：以为 `@dec` 一定会产生“代理”并带来额外调用开销。其实装饰器完全可以零开销（直接返回原对象）——是否包装，取决于你写不写 `wrapper`。

39.5 Using and Defining Decorators (使用与定义装饰器)

英文原文

Depending on your job description, you might encounter decorators as a user or a provider. As we've seen, Python itself comes with built-in decorators that have specialized roles—static and class method declaration, property creation, and more. In addition, many popular Python toolkits include decorators to perform tasks such as managing database or user-interface logic. In such cases, we can get by without knowing how the decorators are coded. For more general tasks, programmers can code arbitrary decorators of their own. For example, function decorators may be used to augment functions with code that adds call tracing or logging, caches call results, performs argument validity testing during debugging, times calls made to functions for optimization, and so on. Any behavior you can imagine adding to—really, wrapping around—a function call is a candidate for custom function decor

ators. On the other hand, function decorators are designed to augment only a specific function or method call, not an entire object interface. Class decorators fill the latter role better—because they can intercept instance-creation calls, they can be used to implement arbitrary object interface augmentation or management tasks. For example, custom class decorators can trace, validate, or otherwise augment every attribute reference made for an object. They can also be used to implement proxy objects, singleton classes, and other common coding patterns. In fact, you'll find that many class decorators are a prime application of the delegation coding pattern we met in Chapter 31.

中文翻译

取决于你的职位描述，你可能以“使用者”或“提供者”的身份遇到装饰器。正如我们所看到的，Python 自带一些具有专门角色的内建装饰器——静态方法（`staticmethod`）与类方法（`classmethod`）声明、`property` 创建等等。此外，许多流行的 Python 工具包也包含装饰器，用来执行诸如管理数据库或用户界面逻辑之类的任务。在这些情况下，我们即使不知道装饰器是怎么编码的，也能顺利使用。对于更一般的任务，程序员可以编写属于自己的任意装饰器。例如，函数装饰器可以用来增强函数：添加调用跟踪或日志记录代码、缓存调用结果、在调试期间执行参数有效性测试、为优化目的而给函数调用计时等等。任何你能想象到可以附加在——准确说是“包裹在”——函数调用周围的行为，都是自定义函数装饰器的候选。另一方面，函数装饰器设计上只能

增强某一个具体的函数或方法调用，而不是整个对象接口（object interface）。类装饰器更适合填补后者的角色——因为类装饰器能拦截实例创建调用，所以它们可以用来实现任意的对象接口增强或管理任务。例如，自定义类装饰器可以跟踪、校验或以其他方式增强对对象所做的每一次属性引用。它们还可以用来实现代理对象、单例类（singleton classes）以及其他常见编码模式。事实上，你会发现许多类装饰器正是我们在第 31 章遇到的委托（delegation）编码模式的典型应用。

深度理解

- 核心概念：本节界定了装饰器的两大适用半径——函数装饰器管"单点调用"，类装饰器管"整体接口"。
- 底层实现：管整体接口靠的是 getattr 钩子：包装器实例上任何未定义属性的访问都会落入 getattr，从而被转交给被包裹对象。
- 设计原因：函数调用只是一次执行，而对象接口是无数次属性访问。拦截调用只需一次重绑定；拦截接口需要持续的代理层，因此用类装饰器更顺手。
- 解决什么实际问题：单例、代理对象、接口跟踪、数据库 ORM 中"一个类对应一张表"的声明、UI 框架的事件绑定——这些都是类装饰器的主场。
- 初学者误区：以为类装饰器只能"装饰用户自定义类"。其实只要包装器足够通用，连 list、dict 等内置类型也能被装饰（第 31 章的委托模式已示范）。

39.6 Why Decorators? (为什么使用装饰器?)

英文原文

Like many advanced Python tools, decorators are never required from a purely technical perspective: we can often implement their functionality instead using simple helper function calls or other techniques. And at a base level, we can always manually code the name rebinding that decorators perform automatically. That said, decorators provide an explicit syntax for such tasks, which makes intent clearer, can minimize augmentation code redundancy, and may help ensure correct API usage: Decorators have a very explicit syntax, which makes them easier to spot than helper function calls that may be arbitrarily far removed from the subject functions or classes. Decorators are applied once when the subject function or class is defined; it's not necessary to add extra code at every call to the class or function, which may have to be changed in the future. Because of both of the prior points, decorators make it less likely that a user of an API will forget to augment a function or class according to API requirements. In other words, beyond their technical model, decorators offer some advantages in terms of both code maintenance and consistency. Moreover, as structuring tools, decorators naturally foster encapsulation.

n of code, which reduces redundancy and makes future changes easier. Like most tools, decorators have some potential drawbacks, too—when they insert wrapper logic, they can alter the types of the decorated objects, and they may incur extra calls when used as call or interface proxies. On the other hand, the same considerations apply to any technique that adds wrapping logic to objects. We'll explore these trade-offs in the context of real code later in this chapter. Although the choice to use decorators is ultimately subjective, their advantages are compelling enough to have escalated them to common practice in the Python world. To help you decide for yourself, let's turn to the details.

中文翻译

与许多高级 Python 工具一样，从纯粹的技术角度看，装饰器从来不是必需的：我们常常可以用简单的辅助函数调用或其他技巧来实现相同的功能。在最底层，我们也总能手动写出装饰器自动执行的那些名字重绑定。话虽如此，装饰器为这类任务提供了显式的语法，这让意图更清晰、能减少增强代码的冗余（code redundancy），并有助于确保 API 的正确使用：装饰器拥有非常显式（explicit）的语法，比那些可能离被装饰函数或类任意遥远的辅助函数调用更容易被发现。装饰器在被装饰的函数或类定义时只应用一次；不需要在被装饰类或函数的每一次调用处添加额外代码——那些调用点将来可能还要修改。由于以上两点，装饰器使得 API 的使用者不太可能忘记按照 API 的要求去增强某个函数

或类。换言之，除了技术模型之外，装饰器在代码维护性与一致性上都带来一些好处。此外，作为结构化工具，装饰器还自然而然地促成了代码的封装（encapsulation），这减少了冗余，也让将来的修改更容易。与大多数工具一样，装饰器也有一些潜在的弊端（drawbacks）——当它们插入包装器逻辑时，会改变被装饰对象的类型；当它们用作调用代理或接口代理时，可能带来额外的调用开销。另一方面，任何为对象添加包装逻辑的技术都有同样的考量。我们将在本章稍后的真实代码中审视这些权衡（trade-offs）。虽然是否使用装饰器最终是主观选择，但它们的优势已强大到足以让它们升级为 Python 世界中的常见实践。为了帮你做出自己的判断，让我们进入细节。

深度理解

·核心概念：本节回答“为什么不直接写 `f = dec(f)` 或封装辅助函数”——答案是显式性（explicitness）与一次应用。装饰器把行为声明放在定义旁，读者无需在代码中四处追查调用点。

·底层实现：语法层面，`@dec` 与手动 `f = dec(f)` 的字节码几乎完全一样；区别在于语法强调了“声明位置”和“单次应用”。

·设计思想：这是 Python 哲学“显式优于隐式”（Explicit is better than implicit）的直接落地，同时兼顾“最小冗余”——一次定义、处处生效。

·解决的实际问题：框架 API 的完整性——让“忘记增强”成为小概率事件；也让“把某个函数注册到某条路由”成为一种声明式表达。

·初学者误区：把"类型改变"的弊端想得过重。确实 `isinstance(f, types.FunctionType)` 在装饰后可能为 `False`，但在绝大多数"以调用来使用"的场景里无碍。本节特意预告了这个权衡，后面还会用 `class`、`isinstance` 演示它。

39.7 DECORATORS VERSUS MACROS (装饰器对比宏)

英文原文

Python's decorators bear similarities to what some call aspect-oriented programming in other languages—code inserted to run automatically before or after a function call runs. Their syntax also very closely resembles (and is likely borrowed from) Java's annotations, though Python's model may be considered more flexible and general. Some liken decorators to macros too, but this isn't entirely apt and can be misleading. Macros, like C's `#define` preprocessor directive, are associated with textual replacement and expansion and designed for generating code. By contrast, Python's decorators are a runtime operation based upon name rebinding, callable objects, and often, proxies. While the two may have use cases that sometimes overlap, decorators and macros are fundamentally different in scope, implementation, and coding patterns. Comparing the two seems akin to comparing Python's `import` with a C `#include`, which similarly confuses a run

time object-based operation with text insertion. Of course, the term macro has also been diluted over time—to some, it now can also refer to any canned series of steps or procedure—and users of other languages might find the analogy to decorators useful anyhow. But they should also keep in mind that decorators are about callable objects managing callable objects, not text expansion. Python tends to be best understood and used in terms of Python idioms.

中文翻译

Python 的装饰器与某些语言中所谓的面向切面编程 (aspect-oriented programming) 有相似之处——即在函数调用运行之前或之后自动插入执行的代码。它们的语法与 Java 的注解 (annotations) 也非常相似 (很可能是借鉴而来), 不过 Python 的模型可以被认为更灵活、更通用。也有人把装饰器比作宏 (macros), 但这个比喻并不完全恰当, 还可能产生误导。宏, 如 C 语言的 `#define` 预处理器指令, 与文本替换 (textual replacement) 和展开 (expansion) 相关, 其设计目的是生成代码。相比之下, Python 装饰器是运行时 (runtime) 操作——基于名字重绑定、可调用对象, 并且往往基于代理。虽然两者的用例有时重叠, 但装饰器与宏在范围、实现和编码模式上有着根本性的不同。把两者相提并论, 就像把 Python 的 `import` 与 C 的 `#include` 相比——这同样混淆了“基于对象的运行时操作”与“文本插入”。当然, “宏”这个术语随着时间推移已被稀释——对某些人来说, 它现在也可以指任何成套的固定步骤或流程——而且其他语言的用户也许仍会发现把它类比为装饰

器有些用处。但他们也应记住：装饰器是"可调用对象管理可调用对象"，而不是文本展开。Python 最好用 Python 自己的习语来理解和运用。

深度理解

·核心概念：装饰器是运行时基于对象的操作（运行期发生的名字重绑定），而宏是编译前基于文本的操作（字符串替换）。二者分属两个世界。

·底层实现：C 的 `#define` 在预处理阶段就完成了文本替换，编译器看到的已是替换后的代码；Python 的 `@dec` 则在程序运行、模块导入时以普通函数调用方式执行。

·设计原因：Lutz 指出，若把装饰器当宏理解，容易误以为装饰器"展开成代码文本"。实际它只是调用了 callable，这与 Python 一贯的"一切皆对象"哲学一致。

·解决什么实际问题：这个区分帮助开发者选型——需要"生成代码"用宏（或代码生成器），需要"运行时包装行为"用装饰器。Java 注解与 Python 装饰器也类似，但 Python 模型更通用（注解常只是元数据，装饰器则真正包裹行为）。

·初学者误区：把 `@decorator` 想象成"在被装饰函数里粘贴了一段代码"。其实装饰器运行时包装与文本插入完全不同；用 Python 习语思考 Python，而不是用 C 的思维套用。

39.8 The Basics (基础)

英文原文

Let's get started with a first-pass look at decoration behavior from an abstract perspective. We'll write real and more substantial code soon, but since most of the magic of decorators boils down to an automatic rebinding operation, it's important to understand this mapping first—for both functions and classes.

中文翻译

让我们先从抽象视角第一次粗略打量装饰行为。我们很快就会写出真实而更充实的代码，但由于装饰器的大部分“魔法”归根到底是一次自动重绑定（automatic rebinding）操作，先弄清这层映射很重要——对函数和类都是如此。

深度理解

- 核心概念：本节是先导——一切装饰器行为都可归结为“名字重绑定”，理解这个映射是理解整章的前提。
- 底层实现：重绑定发生在定义语句的结尾，编译器生成的字节码顺序是“创建对象 → 调用装饰器 → STORENAME 赋回原名字”。
- 设计原因：先抽象地感知“映射”，再进入真实代码，能避免被各种编码细节（闭包、类、参数）冲昏头脑。
- 解决什么实际问题：为后面所有示例提供一个统一的思维模型——无论装饰器多花哨，最终都能还原成一次赋值。

·初学者误区：急于写装饰器代码，却忽略了"这只是一次重绑定"这一根本心智模型，导致遇到额外参数、嵌套时就迷路。

39.9 Function Decorator Basics (函数装饰器基础)

英文原文

As previewed earlier in this book, function decorators are largely just syntactic "sugar" that runs one function through another at the end of a `def` statement and rebinds the original function name to the result.

中文翻译

正如本书此前预览过的，函数装饰器在很大程度上只是语法"糖"——在 `def` 语句末尾把一个函数送入另一个函数处理，并把原函数名字重绑定到结果上。

深度理解

- 核心概念：函数装饰器 = 在 `def` 末尾执行 `func = decorator(func)` 的语法糖。
- 底层实现：decorator 必须是一元可调用对象（只收一个参数：被装饰函数），其返回值必须是可调用对象，且参数个数要能匹配之后的调用。
- 设计原因：语法糖的目的不是增加能力，而是提高可读性与一致性，让"增强"意图直接写在定义旁边。

·解决什么实际问题：让"计时、日志、缓存"等增强逻辑不需要改函数体，也不需要每处调用点额外包函数。

·初学者误区：以为装饰器能任意改变函数的参数签名。其实 decorator 返回的 callable 要与之后调用时的参数形态匹配，否则调用会失败。

代码分析

```
# Usage
@decorator          #
def F(arg): ...
F(99)               #

#
def F(arg): ...
F = decorator(F)    #
F(99)               # decorator(F)(99)
```

解释器处理顺序：编译 def F 生成函数对象 → 调用 decorator(F) → 把返回值赋回 F。此后每次 F(99) 调用的都是装饰器返回的对象，可能是 wrapper，也可能是原函数本身。书里还给出关键一行注释语句：

```
func(6, 7)          # @decorator func →
decorator(func)(6, 7) #
```

设计思想：装饰只执行一次（定义时），之后每次调用都走包装层；若装饰器返回原函数，则调用等价于没装饰。

Implementation (实现)

英文原文

A decorator itself is a callable that returns a callable. That is, it returns the object to be called later when the decorated function is invoked through its original name—either a wrapper object to intercept later calls or the original function augmented in some way. In fact, decorators can be any type of callable and return any type of callable: any combination of functions and classes may be used, though some are better suited to certain contexts. For example, to tap into the decoration protocol in order to manage a function just after it is created, we might code a decorator of this form: Exactly as coded, this simply adds a post-creation step to function definition: the original decorated function is assigned back to its name. Such a structure might be used to register a function to an API, initialize function attributes, and so on. In more typical use, to insert logic that intercepts later calls to a function, we might code a decorator to return a different object than the original function—a proxy for later calls: This decorator is invoked at decoration time, and the callable it returns is invoked when the original function name is later called. The decorator itself receives the decorated function; the callable returned receives whatever arguments are later passed to the decorated function's name. When coded properly, this works the same for class-level methods: the implied instance

object just shows up in the first argument of the returned callable. In skeleton terms, here's one common coding pattern that captures this idea—the decorator returns a wrapper that retains the original function in an enclosing scope: When the name `func` is later called, it really invokes the wrapper function returned by decorator; the wrapper function can then run the original `func` because it is still available in an enclosing scope. When coded this way, each decorated function produces a new scope to retain state. To do the same with classes, we can overload the call operation and use instance attributes instead of enclosing scopes: When the name `func` is later called now, it really invokes the call operator-overloading method of the instance created by calling the decorator class; the call method can then run the original `func` because it is still available in an instance attribute `self.func`. When coded this way, each decorated function produces a new instance to retain state.

中文翻译

装饰器本身是一个“返回可调对象的可调对象（callable that returns a callable）”。也就是说，它返回的是“当被装饰函数经由原名调用时，之后要调用的那个对象”——可以是一个拦截后续调用的包装器对象，也可以是以某种方式增强后的原函数。事实上，装饰器可以是任意类型的可调对象，也可以返回任意类型的可调对象：函数与类的任何组合皆可，只是某些组合更适合特定场景。例如，为了接入

装饰协议、在函数刚创建后就管理它，我们可以编写这样形式的装饰器：

```
> python > def decorator(F): > # 在这里
对函数 F 进行处理> return F # 返回原函数> @decorator
> def func(): ... # func = decorator(func) > 正是因此，
它只是给函数定义添了一个"创建后处理步骤"：被装饰的原
函数被赋回它自己的名字。这样的结构可以用来把函数注册
到一个 API、初始化函数属性等等。在更典型的使用中，为
了插入拦截后续调用的逻辑，我们可以让装饰器返回一个与
原函数不同的对象——一个针对后续调用的代理：(default
) python def decorator(F): # 保存或使用函数 F # 返回一个
不同的可调用对象：嵌套 def、带 call 的类实例等@dec
orator def func(): ... # func = decorator(func) 这个装饰
器在"装饰时刻"被调用，它返回的可调用对象则在"原函数
名字被后续调用"时被调用。装饰器本身收到被装饰的函数
；返回的可调用对象收到之后传给被装饰函数名字的各种参
数。只要编码得当，这对类层次的方法（methods）同样
成立——隐藏的实例对象会自然地出现在返回可调用对象
的第一个参数中。用骨架代码来说，这里有一个抓住该要点
的常用编码模式——装饰器返回一个保留原函数于闭包作
用域的包装器：python def decorator(F): # 在 @ 装饰时
执行def wrapper(args): # 在包装函数被调用时执行# 使用
F 与 args return F(args) # F 调用原函数return wrapper
@decorator # func = decorator(func) def func(x, y): ...
# func 被传入装饰器的 F func(6, 7) # 6, 7 被传入 wrapp
er 的 args 当名字 func 之后被调用时，它实际调用的是 d
ecorator 返回的 wrapper 函数；由于原函数 func 还在外
层作用域（enclosing scope）里，wrapper 可以运行它。
按这种方式编码，每个被装饰的函数都会产生一个新的作用
```

域来保留状态。若想用类达到同样的效果，我们可以重载调用操作（call），用实例属性代替外层作用域：python class decorator: def init(self, func): # 在 @ 装饰时执行 self.func = func # 保存原函数到实例属性 def call(self, args): # 在包装函数被调用时执行 # 使用 self.func 与 args return self.func(args) # 调用原函数 @decorator # func = decorator(func) def func(x, y): ... # func 被传入 init func(6, 7) # 6, 7 被传入 call 的 args 现在，当名字 func 之后被调用时，它实际调用的是 decorator 类实例的 call 运算符重载方法；由于原函数 func 还能在实例属性（instance attribute） self.func 里取到，call 就能运行它。按这种方式编码，每个被装饰的函数都会产生一个新实例来保留状态。

深度理解

- 核心概念：装饰器是"返回可调用对象的可调用对象"。最核心的编码选择是：用什么外壳包住原函数——嵌套函数的闭包作用域，还是类的实例属性。
- 底层实现：闭包方案依赖"自由变量 + 单元格（cell）"来保留 F；类方案依赖实例的 dict 保存 self.func。二者都是状态保存的手段。
- 设计原因：闭包代码量少、写法直观；类则把状态显式挂在属性上、便于从外部访问。书里两种都示范，正是让读者权衡。
- 解决什么实际问题：装饰器内部需要"记住原函数"——闭包与实例属性是两个主流记忆载体。
- 初学者误区：以为 call 里的 self 是指被装饰类实例。不——它指的是装饰器类自己的实例；这一点在装饰方法时

会引发著名的"方法陷阱" (Class Pitfall) 。

代码分析

上面两幅代码骨架，配合解释器执行路径讲解：

```
def decorator(F):          # ① @ F
    def wrapper(*args):    # ②
        return F(*args)    # ④ cell F
    return wrapper         # ③ wrapper func
func(6, 7)                 # ⑤ wrapper(6, 7)
```

类的版本把 ② 拆成 init (保存 self.func) 与 call (转发调用)，状态从"闭包 cell"变成了"实例属性"。

设计思想：装饰器不直接处理业务逻辑，它只做"接管名字 + 转发原函数"，从而把增强逻辑与业务逻辑解耦。

39.10 Supporting method decoration (支持方法装饰)

英文原文

One subtle point about the prior class-based coding is that while it works to intercept simple function calls, it does not quite work when applied to class-level method functions: When coded this way, a problem arises when the decorated method is referenced later. Because the method name is rebound to an instance of the decorator class, the instance of the subject class (class C) is never passed to the call. The decorator object retains the original method function, but it has n

o instance to pass to it. To support both accessible functions and methods, the nested function form works better: When coded this way, wrapper receives the C class instance in its first argument, so it can dispatch to the original method and access its state information. Technically, this nested-function version works because Python only creates a bound method object and passes the subject class instance to the was self argument when a method attribute references a simple function; when a method attribute references an instance of a callable class instead, only that callable class's instance is passed to self. So you'll need to make sure your decorators are coded accordingly. Also note that nested functions are perhaps the most straightforward way to support decoration of both functions and methods, but not necessarily the only way. The prior chapter's descriptors, for example, receive both the descriptor-class and subject-class instance when called. Though more complex, later in this chapter you'll see how this tool can be leveraged to decorate functions and methods.

中文翻译

关于前面基于类实现的编码，有一个微妙点：它能拦截简单的函数调用，但当它应用到类级别的方法时就不太管用了：(Code: class decorator with init/call) 按这种方式编码，被装饰的方法后来被调用时会出问题。因为方法名现在被重新绑定到装饰器类的一个实例上，主题类的实例（class C 的

实例) 永远不会被传入调用参数。装饰器对象虽然保留着原方法函数, 但它没有任何实例可传给这个方法。为了同时支持函数与方法, 嵌套函数形式工作得更好: 按这种方式编码时, wrapper 能在第一个参数里收到 class C 的实例, 因此它可以派发到原方法并访问其状态信息。从技术上讲, 嵌套函数版本之所以可行, 是因为 Python 只有一个方法属性引用的是简单函数时, 才会创建绑定方法 (bound method) 对象并把主题实例传给 self; 当一个方法属性引用的是可调用类的实例时, 传给 self 的反而是这个可调用类自己的实例。所以你必须确保你的装饰器编码得当。还要注意: 嵌套函数大概是支持"同时装饰函数和方法"最直接的方式, 但不一定是唯一方式。例如, 前一章的描述符 (descriptors) 在调用时能同时收到描述符类实例和主题实例。虽然更复杂, 但本章稍后你会看到如何借助这个工具去装饰函数与方法。

深度理解

- 核心概念: class-based 装饰器只适合装饰"裸函数"; 一旦装饰类里的方法, self 就会被装饰器实例霸占, 主题类 C 的实例彻底丢失。
- 底层实现: Python 把"方法调用"实现为两步——第一步对 C.method 做属性解析 (attribute lookup)。如果属性是简单函数, 就包装成绑定了实例的 bound method; 如果属性是可调用类的实例, 则不绑定, 直接把该实例当可调用对象调用。
- 设计原因: 这个差异是 Python 属性协议的内建行为, 不是 bug; 理解它才能解释为什么"类封装装饰器"会踩坑。

·解决的实际问题：写装饰器时如果希望"函数和方法通用"，优先用嵌套函数（把 self 留在 args 里），而不是用带 call 的可调用类。

·初学者误区：看到 `tracer.call(self, args)` 就以为 self 是被装饰的对象——实则 self 是装饰器（tracer）实例本身。

最终，本章前段那句"不必担心"的总结在这里被打脸：类方案并不是万能答案。后面"Class Pitfall: Decorating Methods"整节将展开演示其崩溃过程，这里先记住方向。

（本节对应书籍中的三幅简代码示意，含义与上方骨架相同，故浓缩于此演示。）

39.11 Class Decorator Basics（类装饰器基础）

英文原文

Function decorators proved so useful that the model was extended to allow class decoration. They were initially resisted because of role overlap with the next chapter's metaclasses; in the end, though, they were adopted because they provide a simpler way to achieve many of the same goals. Class decorators are strongly related to function decorators; in fact, they use the same syntax and very similar coding patterns. Rather than wrapping individual functions or methods, th

ough, class decorators are a way to manage classes or wrap up instance-creation calls with extra logic that manages or augments instances created from a class. In the latter role, they may manage full object interfaces instead of a single callable object.

中文翻译

函数装饰器实在太有用了，于是该模型被扩展到了类上。类装饰器最初还因为与下一章的元类（metaclasses）角色重叠而遭到抵制；不过最终它们还是被采纳了，因为它们为实现许多同样的目标提供了更简单的方式。类装饰器与函数装饰器关系密切；事实上，它们使用同样的语法和非常相似的编码模式。不过，类装饰器不是去包装单个函数或方法，而是一种管理类的方式，或者说是用一种额外的逻辑把"实例创建调用"包装起来，去管理或增强由某类创建出来的实例。在后一种角色里，它们管理的是完整的对象接口，而不只是一个可调用对象。

深度理解

- 核心概念：类装饰器 = 在 class 语句末尾执行 `C = decorator(C)` 的语法糖。它管理的对象是"类"及"类的实例"。
- 底层实现：与元类在"类创建过程结尾"运行类似，类装饰器也在类创建后立即运行；但类装饰器不需要 type 介入创建过程，因此更轻量。
- 设计原因：Lutz 提到历史——类装饰器因与元类职责重叠曾被抗拒。最终因其"轻量、简单"而被纳入：能做元类 80% 的事，却不需要理解 metaclass 机制。

·解决什么实际问题：单例、代理、接口跟踪、属性隐私校验——这些需要“管住整个类”的需求，用类装饰器一行 @ 即可。

·初学者误区：以为类装饰器只能“装饰”，在实例化时失效。实际恰恰相反——C() 会触发装饰器返回的 callable，实例化自己也已被接管。

代码分析

```
# Usage
@decorator          #
class C: ...
x = C(99)           #

#
class C: ...
C = decorator(C)    #
x = C(99)           # decorator(C)(99)
```

解释器处理顺序：编译 class C 生成类对象 → 调用 decorator(C) → 把返回结果赋回 C。此后 C(...) 触发的实际是装饰器返回的可调用对象，它“可能调用也可能不调用”原类。设计思想：把“实例创建”纳入增强链条——一次装饰、所有实例统一接管。

39.12 Implementation (实现)

英文原文

New class decorators are coded with many of the same techniques used for function decorators, though

some may involve two levels of augmentation—to manage both instance-construction calls as well as instance-interface access. Because a class decorator is also a callable that returns a callable, most combinations of functions and classes suffice. However it's coded, the decorator's result is what runs when an instance is later created. For example, to simply manage a class just after it is created, return the original class itself. To instead insert a wrapper layer that intercepts later instance-creation calls, return a different callable object. To see how this works at the interface level, consider the following example, which inserts an object that intercepts undefined attributes of a class instance: In this example, the decorator rebinds the class name to another class, which retains the original class in an enclosing scope and creates and embeds an instance of the original class when it's called. When an attribute is later fetched from the instance, it is intercepted by the wrapper's `getattr` and delegated to the embedded instance of the original class. Moreover, each decorated class creates a new scope, which remembers the original class. We'll flesh out this example into some more useful code later in this chapter. Like function decorators, class decorators are commonly coded as either closure (a.k.a. "factory") functions that create and return callables, classes that use `__init__` or `__call__` methods to intercept call operations, or some combination thereof. Closure functions typically retain state in

enclosing-scope references, and classes retain state in attributes.

中文翻译

新的类装饰器用很多与函数装饰器相同的技巧来编写，尽管有些可能涉及两级增强（two levels of augmentation）——既要管理“实例构造调用”，又要管理“实例的接口访问”。因为类装饰器同样是一个“返回可调用对象的可调用对象”，所以函数与类的大多数组合都够用。无论怎么编码，装饰器的返回结果就是“之后创建实例时运行的东西”。例如，若只是想在被创建的类刚完成时管理它，直接返回原类本身即可。若想插入一层拦截后续实例创建调用的包装器，则返回另一个不同的可调用对象。为了看看它在接口层次上是如何工作的，考虑下面这个例子——它插入一个对象来拦截类实例的（未定义）属性：(Code: `def decorator(cls) 返回内部 class Wrapper, 用 getattr 委托`)在这个例子中，装饰器把类名字重绑定到另一个类上，这类把原类保留在外层作用域里，并在自己被调用时创建、嵌入一个原类的实例。之后当从实例上抓取某个属性时，该访问会被包装器的 `getattr` 拦截，并委托（delegate）给被嵌入的原类实例。此外，每个被装饰的类都会产生一个新作用域，记住的正是原类。我们稍后会把这个例子充实成更有用的代码。与函数装饰器一样，类装饰器通常写成闭包（closure，也就是“工厂 factory”）函数——创建并返回可调用对象，或使用 `init/call` 方法拦截调用操作的类，或是二者的组合。闭包函数通常在外层作用域引用中保留状态，类则在属性中保留状态。

深度理解

- 核心概念：类装饰器可能涉及两级增强——"实例构造"与"实例接口访问"分别被管理，对应两个不同的钩子时刻。
- 底层实现：用 class Wrapper 的方案里，Wrapper.init 负责创建并嵌入原类实例；Wrapper.getattr 负责把外部属性访问委托给嵌入实例。原类保存在 Tracer/decorator 函数的外层作用域里。
- 设计原因：用函数工厂 + 嵌套类，能让"每个被装饰类"各有一个独立的 Wrapper 类与独立作用域，从而支持多实例、多类互不干扰。
- 解决什么实际问题：属性级代理、接口跟踪、单例、访问校验都需要"在实例化的同时接管接口"，本节给出了最小可行示范。
- 初学者误区：把"被装饰的类"与"装饰器返回的类"混为一谈。被装饰的类被嵌入到了返回类的实例内部，x 前端其实是 Wrapper 实例。

代码分析

```
def decorator(cls):                                # @
    class Wrapper:
        def __init__(self, *args):                #
            self.wrapped = cls(*args)             #
        def __getattr__(self, name):               #
            return getattr(self.wrapped, name)    #
    return Wrapper                                  #

@decorator
class C:
```

```
def __init__(self, x, y):
    self.attr = 'hack'          # Wrapper.__init__ cls(...)

x = C(6, 7)                    # Wrapper(6, 7) __init__ cls(6, 7)
print(x.attr)                  # Wrapper.__getattr__ "hack"
```

解释器执行链：x = C(...) → 名字 C 已指向 Wrapper 类
 → Wrapper.init → 内部 cls(args) 调用真正的原类创建
 实例 → 嵌入 self.wrapped。之后每次 x.attr 都会先落在
 Wrapper.getattr（因为 attr 不是 Wrapper 自己的属性
 名），而后被 getattr 委托给 self.wrapped。设计思想：
 接口代理 = 拦截属性访问钩子（getattr）+ 转发（geta
 ttr），这两者的结合是可复制的委托模式。

Supporting multiple instances（支持多个实例）

英文原文

As for function decorators, some callable-type combinations work better for class decorators than others. Consider the following invalid alternative to the class decorator of the prior example: This code handles multiple decorated classes (each makes a new Decorator instance) and intercepts instance-creation calls (each runs call). Unlike the prior version, however, this version fails to handle multiple instances of a given class—each instance-creation call overwrites the prior saved instance. The prior version does support multiple instances because each instance-creation call makes a new independent wrapper object. More generally,

either of the following patterns supports multiple wrapped instances: a factory function that returns the class itself; or a function that returns a wrapper class intended by calling the original class.

中文翻译

与函数装饰器的情况一样，某些“可调用类型组合”在类装饰器上比别的组合更好用。下面这段是前一示例类装饰器的一个有毛病的替代写法：(Code: class Decorator with init/call/getattr, 单实例保存)这段代码能处理多个被装饰类（每个被装饰类都新建一个 Decorator 实例），也能拦截实例创建调用（每次都运行 call）。不过与前一版本不同的是，这一版处理不了“同一个类的多个实例”——每次实例创建调用都会覆盖之前保存的实例。前一版本之所以支持多实例，是因为每次实例创建调用都会产生一个新的独立 wrapper 对象。更一般地说，下面两种模式支持多个被包装实例：工厂函数（返回一个类，该类在构造时嵌入原类实例），或返回“被调用时创建并返回包装器”的函数的模式。

深度理解

- 核心概念：“每类一个装饰器实例，还是每类实例一个 wrapper？”——状态保存在哪里，决定了能否支持多实例。
- 底层实现：class Decorator 方案里 self.wrapped 是装饰器实例的单个属性，每次 call 都会覆盖它；def decorator + class Wrapper 方案每次调用都 new 一个 wrapper 实例，各自持有独立 self.wrapped。
- 设计原因：这是“状态保存策略”的经典教训：想把状态按

"实例"粒度存储，状态就必须属于被创建的 wrapper 实例而非共享的装饰器对象。

·解决什么实际问题：写通用类装饰器时，务必弄清"你的状态是 per-class 还是 per-instance"，否则会出现莫名其妙的数据串扰。

·初学者误区：只要代码能跑就以为正确——本例中 bob 与 sue 共享同一 wrapped，第二个实例悄悄覆盖第一个，这种 bug 极其隐蔽。

代码分析

```
# Decorator __call__ self.wrapped
class Decorator:
    def __init__(self, C):          # @
        self.C = C
    def __call__(self, *args):      #
        self.wrapped = self.C(*args) #
        return self
    def __getattr__(self, attrname):
        return getattr(self.wrapped, attrname)

@Decorator
class C: ...
x = C()  # self.wrapped = 1
y = C()  # self.wrapped = 2x

# Wrapper new Wrapper
def decorator(C):
    class Wrapper:
        def __init__(self, *args):
            self.wrapped = C(*args)  # wrapped
    return Wrapper

# Wrapper
```

```
def decorator(C):  
    def onCall(*args):  
        return Wrapper(C(*args))  
    return onCall
```

设计思想：状态粒度的选择直接决定行为正确性。这是本章反复出现的"状态保存 (state retention)"主线的第一次爆雷，后文还会多次遇见。

39.13 Decorator Nesting (装饰器嵌套)

英文原文

Sometimes, one decorator isn't enough. For instance, suppose you've coded two function decorators to be used during development—one to test argument types before function calls and another to test return value types after function calls. You can use either independently, but what to do if you want to employ both on a single function? What you really need is a way to nest the two, such that the result of one decorator is the function decorated by the other. It's irrelevant which is nested, as long as both steps run on later calls. To support multiple nested steps of augmentation this way, decorator syntax allows you to add multiple layers of wrapper logic to a decorated function or method. When this feature is used, each decorator must appear on a line of its own. Decorator syntax of this form: @A / @B / @C / def f(...) runs the same as the following: def f(...), then f = A(B(C(f))). Here, the origin

al function is passed through three different decorators, and the resulting callable object is assigned back to the original name. Each decorator processes the result of the prior, which may be the original function or an inserted wrapper. If all the decorators insert wrappers, the net effect stacks them: when the original function name is called, three different layers of wrapping object logic will be invoked to augment the original function in three different ways. The last decorator listed is the first applied and, thus, the most deeply nested when the original function name is later called. Just as for functions, multiple class decorators result in multiple nested function calls and possibly multiple levels and steps of wrapper logic around instance-creation calls. With wrappers, when an instance of the original C class is finally requested, the call is redirected to the wrapping layer objects, which may have arbitrarily different roles—they might trace and validate attribute access for example, and both steps would be run in turn on later requests.

中文翻译

有时候一个装饰器不够用。比如你编写了两个开发期函数装饰器——一个在函数调用前测试参数类型，另一个在函数调用后测试返回值类型。你可以单独用其中任意一个，但如果你想把两者同时作用在一个函数上该怎么办呢？你真正需要的是一种嵌套（nest）两者的方式：让上一个装饰器的结果，成为下一个装饰器要装饰的函数。谁在外谁在内无所谓

，只要两次增强都能在后续调用上运行即可。为了以这种方式支持多步嵌套增强，装饰器语法允许你给同一函数或方法叠加多层包装逻辑。使用这一特性时，每个装饰器都必须独占一行。这种形式的装饰器语法：`@A / @B / @C / 其下`
`def f(...)`，与下面等价：先写 `def f(...)`，再 `f = A(B(C(f)))`。在这里，原函数被依次穿过三个不同的装饰器，最终得到的可调用对象被赋回原名。每个装饰器处理的是前一个装饰器的结果——可能是原函数，也可能是插入的包装器。如果所有装饰器都插入了包装器，净效果就是把它们叠加（stack）起来：当原函数名字被调用时，会依次触发三层包装对象逻辑，从三个不同方面增强原函数。列在最下方的装饰器最先被应用，因此在后续调用时嵌套得最深。与函数类似，多个类装饰器也会产生多重嵌套的函数调用，在实例创建调用周围可能有多层多级的包装逻辑。当真正需要原 C 类的实例时，调用会被改道到各装饰器提供的包装层对象——它们的角色可以千差万别（比如跟踪与校验属性访问），每一次后续请求都会依次走完各层。

深度理解

- 核心概念：装饰器嵌套 = 多个装饰器叠加。映射规则 $f = A(B(C(f)))$ ：自下而上应用（C 先应用、最深），调用时自上而下包装（A 最外层先被触发）。
- 底层实现：这就是嵌套函数调用的普通求值顺序：先求 C(f)、再以结果为参求 B(...)、最后 A(...) 赋回 f。
- 设计原因：单一装饰器只能注入一种增强；叠加允许"参数校验 + 返回值校验 + 日志"这类多横切关注点同时存在。
- 解决什么实际问题：把正交的增强逻辑拆成多个装饰器分

别复用，而不是写一个巨型装饰器而边界复杂。

·初学者误区：记反应用顺序。记住"先贴先内、后贴先触"——@A 在最上，调用时 A 的包装最先执行、最晚交给下一层。

代码分析

```
# ""
def d1(F): return lambda: 'X' + F()    # 'X'
def d2(F): return lambda: 'Y' + F()
def d3(F): return lambda: 'Z' + F()

@d1
@d2
@d3
def func(): return 'hack'              # func = d1(d2(d3(func)))
print(func())                          # "XYZhack"
```

执行链：d3 先包，产生 lambda:'Z'+'hack'; d2(d3(func)) 再包出'Y'+(...); 最终 d1 最外层。调用 func() 时从外向内：lambda:'X' + (lambda:'Y' + (lambda:'Z' + 'hack'))())()，即最外层的 d1 包装先运行，最内层 d3 的包装最后运行、最先返回。设计思想：每个包装器借助闭包记住被包装者 F，可无限叠加；同语法同样适用于类。

39.14 Decorator Arguments (装饰器参数)

英文原文

Both function and class decorators can also seem to take arguments. Really, though, the role of these argu

ments is simpler than it may seem: decorator arguments are passed to a callable that returns the decorator—which in turn returns a callable. By nature, this usually sets up multiple levels of state retention. The following, for instance: `@decorator(A, B) / def F(arg) / F(99)` is automatically mapped into this equivalent form, where `decorator` is a callable that returns the actual decorator. The returned decorator in turn returns the callable that handles later calls to the original function name: `def F(arg): ... / F = decorator(A, B)(F)` (Rebind `F` to result of decorator's return value) / `F(99)` (Essentially calls `decorator(A, B)(F)(99)`) Decorator arguments are resolved before decoration ever occurs, and they are usually used to retain state information for use in later calls. In other words, decorator arguments often imply three levels of callables: a callable to accept the decorator arguments, which returns a callable to serve as the decorator, which returns a callable to handle calls to the original function or class. Each of the three levels may be a function or class, and may retain state in the form of scopes or class attributes. Decorator arguments can be used to provide attribute initialization values, call-trace message labels, attribute names to be validated, and much more—any sort of configuration parameter. A decorator with arguments is sometimes called a "decorator factory," since it creates a decorator that in turn creates a wrapper.

中文翻译

函数装饰器和类装饰器看起来也都能"带参数"。但其实这些参数的角色比看起来更简单：装饰器参数是被传给一个"返回装饰器的可调用对象"的，而这个返回的装饰器又返回一个可调用对象。这样的结构天然会展开多级状态保存。例如下面这段：`@decorator(A, B) / def F(arg): ... / F(99)`自动映射成如下等价形式，其中 `decorator` 是一个返回真正装饰器的可调用对象；返回的装饰器再返回一个可调用，供后续对原函数名的调用使用：`def F(arg): ...; F = decorator(A, B)(F)`（把 `F` 重绑定为 `decorator` 返回值的调用结果）；`F(99)`（本质上调用 `decorator(A, B)(F)(99)`）装饰器参数在装饰发生之前就已解析完成，通常用来保留状态信息供后续调用使用。某种角度看，装饰器参数天生隐含三级可调用对象（three levels of callables）：第一级接受装饰器参数并返回一个"作为装饰器"的可调用，第二级返回一个"处理对原函数或类调用"的可调用。三级中的每一级都可是函数或类，也都可以用作用域或类属性保存状态。装饰器参数可以用来提供属性初始化值、调用跟踪消息的标签（label）、需要校验的属性名等等——任何这类"配置参数"都行。带参数的装饰器有时也被叫作"装饰器工厂（decorator factory）"。

深度理解

- 核心概念：`@decorator(A, B)` 并不是"装饰器带了参数"，而是"工厂函数 `decorator(A, B)` 返回一个真正的装饰器，后者再返回 `wrapper`"。三级嵌套是理解钥匙。
- 底层实现：第一级 `decorator(A, B)` 在 `@` 之前就求值（

参数先解析)；返回的 actualDecorator(F) 在 def 结束时应用；actualDecorator 返回的 wrapper 在每次调用时执行。

·设计原因：配置信息（标签、上下界、姓名）必须在装饰时刻就固定下来，所以要"先接受配置、再接受函数、最后应付调用"。

·解决什么实际问题：让同一个装饰器模板能被不同函数以不同配置复用，比如 @timer(label='fast') 与 @timer(label='slow') 两套计时配置并行存在。

·初学者误区：写嵌套时层数混乱。可以把每层想象成"参数收集器 → 装饰器 → 包装器"三个尽责员，各自只负责一级调用。

代码分析

```
def decorator(A, B):  
    # 1 @ AB  
    def actualDecorator(F):  
        # 2 @ F  
        def wrapper(*args):  
            ...  
            return wrapper  
        return actualDecorator  
  
@decorator(A, B)                                     # decorator(A, B) actualD...  
def F(arg): ...  
F(99)                                                # wrapper(99)
```

设计思想：三级结构把"配置"、"被装饰对象"、"运行期参数"三个不同生命周期分装进三处作用域，互不污染。

39.15 Decorators Manage Functions and Classes, Too (装饰器也能管理函数和类)

英文原文

To wrap up, although much of the rest of this chapter focuses on wrapping later calls to functions and classes, it's important to remember that the decorator mechanism is more general than this—it is simply a protocol for passing functions and classes through any callable immediately after they are created. As such, it can also be used to invoke arbitrary post-creation processing:

```
def decorator(O): / # Augment function or
class O processes / return O @decorator / def F(): ...
# F = decorator(F) @decorator / class C: ... # C = decorator(C)
```

If we return the original decorated object this way instead of a proxy, we are not intercepting later calls—instead we'd run a one-time processed action at definition time. Such decorators might be used to register callable objects to an API, initialize attributes in functions or classes when they are created, and so on. Decorator roles are limited only by your imagination.

中文翻译

小结一下 (To wrap up) :虽然本章后面的主体内容都聚焦于"包装对函数与类的调用", 但要记住: 装饰器机制比这更通用——它其实只是一个"在对象创建后立刻把函数或类交

给任意可调用对象处理一次"的协议。因此它也能用来执行任意的创建后处理：`def decorator(O): # 增强函数或类 O / return O @decorator` 修饰 `def F、@decorator` 修饰 `class C` $\rightarrow F = decorator(F)$ 、 $C = decorator(C)$ 如果我们这样返回原装饰对象而不是代理，就并没有拦截后续调用——而是在定义时刻运行一次"一次性处理"。这种装饰器可以用注册可调用对象到 API、在函数或类创建时初始化属性等等。装饰器的角色只受你的想象力限制。

深度理解

- 核心概念：装饰器协议 = "创建后把对象交给某个 callable 处理一次"。包装调用只是它最常见的用途，不是全部。
- 底层实现：只要 `decorator(O)` 返回的就是 `O` 本身，名字绑定就没有发生替换，调用行为零改动。
- 设计原因：把"注册回调、附加元数据、登记路由"这类创建侧动作也统一到 `@` 语法下，代码更集中、意图更清晰。
- 解决实际问题：ORM 模型注册、事件处理器登记、函数属性初始化——"定义时跑一次"的需求，装饰器顺手就能办。
- 初学者误区：以为装饰器必须返回"别的东西"。返回原对象完全合法，且是"零开销增强"的范本。

39.16 Coding Function Decorators (编写函数装饰器)

英文原文

On to the code. In the rest of this chapter, we are going to study working examples that demonstrate the decorator concepts we've surveyed. This section presents a handful of function decorators in complete form, and the next shows tangible class decorators in action. Following that, we'll close out with two larger case studies that showcase typical decorator roles and code full-scale implementations of class privacy and argument range validation.

中文翻译

进入到代码。在本章剩余部分，我们将研究能用起来实际演示上述装饰器概念的示例。本节以完整的形式给出几个函数装饰器，下一节展示真实可用的类装饰器。在那之后，本章将用两大案例收尾：一个实现"类属性隐私（class privacy）"，另一个实现"参数范围校验（argument range validation）"。

深度理解

- 核心概念：从本节开始，书进入"用代码说话"。四个函数装饰器：跟踪（tracer/timing）、带参 timer、状态保存四方案。
- 底层实现：每个示例都围绕"名字重绑定 + 包装器 + 状态保存"三板斧展开；同一思路反复出现，帮读者提炼共性。
- 设计原因：按"从简单到复杂"组织——先无参装饰，再加

参数；先纯函数，再处理方法陷阱。

·实际问题：这些装饰器本身就是可复用的"通用工具"，直接可移植进读者自己的项目（如 `timerdeco2.py`）。

·初学者误区：只见代码、不看演变。每一版相比上一版只改一处（如加 `kwargs`、加返回、换状态载体），应带着"diff 心态"去读。

39.17 Tracing Function Calls（跟踪函数调用）

英文原文

To get started, let's revive the call tracer example we met in Chapter 32. Example 39-1 defines and applies a function decorator that counts the number of calls made to the decorated function and prints a trace message for each call. Notice how each function decorated with this class will create a new instance with its own saved function object and calls also count. Also observe how the `args` argument is used to pack and unpack arbitrarily many passed-in arguments. This generality enables this decorator to be used to wrap any function with any number of positional arguments; this version doesn't yet work on keyword arguments or class-level methods and returns no results, but we'll fix these shortcomings later in this section. Now, if we import this module's function and test it interacti

vely in a REPL, we get the following kind of behavior. When run, the tracer class saves away the decorated function and intercepts later calls to it in order to add a layer of logic that counts and prints each call. Notice how the total number of calls shows up as an attribute of the decorated function—hack is really a tracer class instance; that's a finding that may have ramifications for programs that do type checking. For function calls, the @ decoration syntax can be more convenient than modifying each call to explode an extra logic level. Consider a nondecorator equivalent. This alternative can be used on any function without the special @ syntax, but it requires extra syntax at each call site—and the call is not guaranteed to be routed through the tracing logic. Although decorators are never, we can always rebind names manually—they are often the most convenient and uniform augmentation point.

中文翻译

作为起点，让我们重新激活第 32 章中遇见过的调用跟踪器（call tracer）示例。示例 39-1 定义并应用了一个函数装饰器：统计被装饰函数的调用次数，并为每次调用打印一条跟踪消息。注意，每个被该类装饰的函数都会创建一个新的实例，拥有自己的函数对象和调用计数。还要注意 args 参数是如何用来打包和解包任意多个传入参数的。这种通用性使这个装饰器能包装任意数量位置参数的任何函数；这版还不能处理关键字参数或类方法，也不返回结果，这些短板我

们稍后在本节修复。现在，如果在 REPL 中交互式地 `import` 这个模块的函数并测试它，我们得到如下行为。运行时，`tracer` 类保存被装饰的函数，并拦截对它的后续调用，从而添加一层"计数并打印每次调用"的逻辑。注意总调用次数显示成了被装饰函数的属性——`hack` 被装饰后其实是一个 `tracer` 类实例；这一事实对做类型检查的程序可能产生暗示意义。对于函数调用，`@` 装饰语法比"在每次调用修改以额外逻辑"更方便。考虑一个非装饰器的等价物：

```
python calls = 0
def tracer(func, args):
    global calls
    calls += 1
    print(f'call {calls} to {func.name}')
    func(args)
def hack(a, b, c):
    # 非装饰函数
    print(a, b, c)
hack(1, 2, 3)
# 普通未跟踪调用：会不会「意外」？
tracer(hack, 1, 2, 3)
# 不用装饰器的专门跟踪调用
这种替代方案能用于任何函数而不需要 @ 语法，但它要求在每个调用点加上额外语法，且无法保证正常调用会经过跟踪层。虽然装饰器从来不是必需的——我们总能手动改名重绑定——它们却常常是最方便、最统一的增强方案。
```

深度理解

- 核心概念：`tracer` 用"类实例 + `call`"实现装饰器，状态藏在 `self.calls`，每装饰一个函数就产生一个新实例，计数器彼此独立。
- 底层实现：装饰后 `hack` 名字指向 `tracer` 实例；每次 `hack(...)` 都触发 `tracer.call`，其内部 `self.calls += 1` 并转发给真正的 `self.func`。
- 设计原因：类的实例属性天然就是"每装饰一份状态"，直观可见，还能被外部访问 (`hack.calls`) 。

·解决什么实际问题：部署期"谁调用了几次、调用参数是什么"——调试、监控、审计的常见需求。REPL 里 `hack(..)` 立刻能看见 `trace` 与计数。

·初学者误区：`print(hack)` 输出 `<decorator1.tracer object ...>` 让不少人意外——这正是"装饰替换了对象"的直接证据，不是 bug。

代码分析

```
# 39-1decorator1.py
class tracer:
    def __init__(self, func):      # @
        self.calls = 0
        self.func = func
    def __call__(self, *args):     #
        self.calls += 1
        print(f'call {self.calls} to {self.func.__name__}')
        self.func(*args)

@tracer
def hack(a, b, c):                # hack = tracer(hack)
    print(a + b + c)             # hack

# REPL
>>> from decorator1 import hack
>>> hack(1, 2, 3)                 # tracer
call 1 to hack
6
>>> hack('a', 'b', 'c')         # __call__
call 2 to hack
abc
>>> hack.calls                   #
2
>>> hack
<decorator1.tracer object at 0x10cafc680>
```

解释器执行链: @tracer 时 tracer.init(hack) 建实例并保存; 此后 hack(...) → Python 调用 tracer.call(hack实例, ...) → 计数、打印、调用 self.func。设计思想: 以 self 为核心的状态对象, 把"计数器 + 原函数"打包在同一修饰对象里, 天然每函数一份。

39.18 Decorator State Retention Options (装饰器状态保存选项)

英文原文

The preceding example raises an important point. Decorators have a variety of options for retaining state information provided at decoration time to be used during later calls to decorated objects. They generally need to support multiple decorated objects and multiple later calls, but there are several ways to implement these goals: instance attributes, global variables, nonlocal closure variables, and function attributes can all be used for retaining state. This topic parallels the initial state coverage in Chapter 17 but can be fleshed out here with class details, and it is so endemic to decorators that it qualifies as a prerequisite. This topic also applies to both function and class decorators, but let's explore it in the narrower function-decorator realm.

中文翻译

前面的例子引出一个重要问题。对于"装饰时刻提供、后续调用时使用"的状态信息，装饰器有各种各样的保存选项。它们通常需要支持多个被装饰对象、多次后续调用，但实现目标有几种途径：实例属性（instance attributes）、全局变量（global variables）、nonlocal 闭包变量（nonlocal closure variables）、函数属性（function attributes）都可以用来保存状态。这个主题与第 17 章的状态保存入门讲解平行，但这里可以用类的细节把问题讲得更饱满；它如此贯穿装饰器始终，几乎算是一个先修知识点。这对函数装饰器和类装饰器都成立，不过本节先在较窄的函数装饰器范围内探索它。

深度理解

- 核心概念：状态保存（state retention）是装饰器的主线难题——"装饰时刻的信息如何活到后续调用"，四大载体：实例属性、全局、nonlocal、函数属性。
- 底层实现：四种载体分别是"对象 dict"、"模块命名空间"、"闭包单元格（cell）"、"函数对象的命名空间"。各自的可见性与每函数隔离性不同。
- 设计原因：没有唯一解——要"每函数独立"还是"全局共享"、要"外部可见"还是"内部私密"，决定了选哪种载体。
- 解决什么实际问题：计数器、累计时间、配置标签、缓存——所有需要在多次调用之间保持可变的量，都得先想清楚存哪。

·初学者误区：在嵌套函数里直接 `+=` 外层变量而报 `UnboundLocalError`，或是用了全局变量导致所有装饰函数共享一个计数器。

State with class-instance attributes (用类实例属性保存状态)

英文原文

As an opening act in the state-retention show, Example 39-2 codes an augmented version of the prior example, which adds support for keyword arguments with syntax and returns the wrapped function's result to support more use cases (for nonlinear readers, we first studied keyword arguments in Chapter 18). Like the original, this uses class instance attributes to save state explicitly. Both the wrapped function and the calls counter are per-instance information—each decoration gets its own copy. When run as a script, notice how the `hack` and `code` functions each have their own calls counter because each decoration creates a new class instance. While useful for decorating functions, this coding scheme still has issues when applied to methods—a shortcoming we'll address in a later revision.

中文翻译

作为状态保存这场大戏的开场，示例 39-2 编写了前一示例

的增强版：为关键字参数加上了语法支持，并返回"被包装函数的结果"以支持更多用例（对非线性读者说明：关键字参数我们最早在第 18 章学过）。与原型一样，它用类实例属性显式保存状态。被包装函数与计数都是"每实例（per-instance）"的信息——每次装饰都得到自己的一份。以脚本运行时可以看到，hack 与 code 两个函数各有自己的计数，因为每次装饰都创建了新的类实例。虽然这个方案装饰函数很实用，但当它应用到方法上时仍有问题——这个短板我们在后面的修订中解决。

(书里这里还讨论了运行输出：call 1 to hack / 6 / call 2 to hack / 15 / call 1 to code / 16 / call 2 to code / 65536。)

代码分析

```
# 39-2decorator_state_classes.py
class tracer:
    #
    def __init__(self, func):    # @
        self.calls = 0          # func
        self.func = func
    def __call__(self, *args, **kwargs):    #
        self.calls += 1
        print(f'call {self.calls} to {self.func.__name__}')
        return self.func(*args, **kwargs) #

@tracer
def hack(a, b, c):              # hack = tracer(hack)
    print(a + b + c)           # tracer.__init__

@tracer
def code(x, y):                 # code = tracer(code)
    print(x ** y)              # code tracer
```

```

if __name__ == '__main__':
    hack(1, 2, 3)          # tracer tracer.__call__
    hack(a=4, b=5, c=6)    # hack
    code(4, 2)             # tracer self.func code
    code(2, y=16)          # self.calls per-decoration

```

改进点：多了 `kwargs`（支持关键字参数）与 `return`（返回原函数结果）。设计思想：装饰器要“透明代理”，就应把原名该收到的参数与返回值都原样转发——加返回值是通用工具的基本修养。

39.19 State with global variables (以全局变量保存状态)

英文原文

For simpler tasks that don't require per-function data, moving state variables out to the global scope, as illustrated by Example 39-3, might suffice. This code still uses an enclosing-scope reference for the original decorated function but pushes the call counter out to the immediate enclosing scope. Unfortunately, moving the counter out to the common shared global scope to allow it to be changed like this also means that it will be shared by every wrapped function. Unlike class instance attributes, global counters are cross-program, not per-function—the counter is incremented for any traced function call. You can tell the difference if you compare this version's output with the prior

version's—the single, shared global call counter is incorrectly updated by calls to every decorated function. Example 39-3. The counter here is incremented for calls to any wrapped function, showing the shared global state problem.

```
python # 示例 39-3: decoratorstateglobals.py calls = 0
def tracer(func): # 状态通过外层作用域与全局变量保存
def wrapper(args, kwargs): # 而不是用类属性
global calls # calls 是全局, 不是每函数一份
calls += 1
```

```
print(f'call {calls} to {func.name}')
return func(args, kwargs)
return wrapper
@tracer
def hack(a, b, c): # 同 hack = tracer(hack)
print(a + b + c)
@tracer
def code(x, y): # 同 code = tracer(code)
print(x y) # 输出会显示 call 3 to code / call 4 to code ——共享计数器的副作用!
```

对照结论: 这里特意给出与示例 39-2 输出的对照——call 1/2 属于 hack, 之后 code 竟从 3 开始计数。 --- ## State with enclosing-scope nonlocals (以外层作用域 nonlocal 保存状态) Shared global state may be what we want in some extreme cases. If we really want a per-function counter, though, we can either use classes as before or make use of closure functions and the nonlocal statement described in Chapter 17. Because this statement allows enclosing function scope variables to be changed, they can serve as per-decoration, changeable data. Example 39-4 demos the basics of this scheme. Now, because enclosing-scope variables are not cross-program globals, each wrapped function gets its own counter gains again, just as for classes an

d attributes. Here's the new output.

```
python # 示例 39-4: decoratorstatenonlocals.py def t
racer(func): # 状态通过外层作用域 + nonlocal 保存 calls
= 0 # 代替类属性或全局变量 def wrapper(args, kwargs):
nonlocal calls # calls 是每函数的, 不是全局的 calls +=
1
```

```
print(f'call {calls} to {func.name}') return func(args, kw
args) return wrapper @tracer def hack(a, b, c): ... @tr
acer def code(x, y): ... # 运行结果与 class 版本完全一致
: hack 与 code 各自计数 (1,2 / 1,2...) 解释器层面: trac
er(func) 执行时, calls 局部于该帧; wrapper 捕获它形成
闭包。wrapper 返回后 tracer 帧销毁, 但闭包单元格仍在
。nonlocal calls += 1 即写入该单元格。设计思想: 用"闭
包 + nonlocal"获得私有、per-decoration 的可变状态。
```

```
--- ## 总结 (State with function attributes) ——以函
数属性保存状态 Finally, you can also avoid globals and
functions classes by making use of function attribute
s for some changeable state instead of nonlocal. As
we saw in Chapters 17 and 19, we can attach arbitrar
y attributes to functions by assignment, with func.attr
= value. Because a factory function makes a new func
tion on each call, its attributes become per-call state.
Moreover, you need to use this technique only for st
ate that must change; enclosing-scope references are
still retained and work normally. To demo, Example 3
9-5 simply uses wrapper.calls for state. It works the s
ame as the preceding nonlocal version because the c
ounter is again per-decorated-function. As we learne
```

d in Chapter 17, this works only because the name wrapper is retained in the enclosing tracer function's scope. When we later increment wrapper.calls, we are not changing the name wrapper itself, so no nonlocal declaration is required. This scheme was almost relegated to a footnote because it is perhaps more obscure than nonlocal and might be probably saved for cases where other schemes don't help. However, function attributes have a substantial advantage: like class instances, they allow access to the saved state from outside the decorator's code; nonlocals can only be seen inside the nested function itself, but function attributes have wider visibility. We will employ them again in an answer to one of the end-of-chapter questions, where their visibility outside callables becomes an asset. As changeable state associated with a context of use, they're equivalent to nonlocals. As usual, choosing from multiple tools is an inherent part of the programming task. Because decorators often imply multiple levels of callables, you can combine functions with enclosing scopes, classes with attributes, and function attributes to achieve a variety of coding structures. As you'll see later, this sometimes will be subtler than you expect—each decorated function should have its own state, and each decorated class may require state both for itself and for each generated instance.

中文翻译

对于不需要每函数数据的简单任务，把状态变量挪到全局作用域可能就足够了，示例 39-3 就是这样做的。这段代码仍用外层作用域引用保存原始被包裹的函数，但把调用计数器推向模块级（全局）。(Code: calls = 0 + 顶层 def tracer(func): 内层 def wrapper 用 global calls) 遗憾的是，把计数器挪到公共的全局作用域以便这样修改，也意味着它会被每一个被包装的函数共享。与类实例属性不同，全局计数器是跨程序的、不是每函数的——任何被跟踪函数的调用都会让计数器自增。对比本版与前版输出即可分辨：这个单一共享的全局计数器被所有被装饰函数的调用错误地更新了。

深度理解 - 核心概念：全局变量的"便宜"来自"粗粒度"——它懒得分家，所有被装饰函数共用一个计数器。

- 底层实现：wrapper 里用 global calls 声明，直通模块级命名空间；闭包只用来保留 func，计数器不受控于装饰。
- 设计原因：有些需求确实要"全局统计"，比如记录"整个程序里被跟踪函数的总调用次数"。
- 解决的实际问题：若目标只是"所有跟踪函数共享一个总计数"，这是最简实现。
- 初学者误区：以为它与示例 39-4 输出应该一样。实际上是全世界共享：运行后被 code 的调用也把 hack 的计数推高了。

代码分析共享的全局状态在个别极端场景下也许正是我们想要的。但如果确实想要"每函数一个计数器"，我们既可以像之前那样用类，也可以利用闭包函数与第 17 章讲过的 nonlocal 语句。由于 nonlocal 允许修改外层函数的局部变量，这些变量就可以充当"每装饰一份、可改变"的数据。示例 39-4 演示了这一方案的基础。现在，由于外层作用域变量不是跨程序的全局量，每个被包装函

数又都拥有自己的计数器了——正如用类与属性时那样。

下面是新输出。#### 深度理解 - 核心概念: nonlocal 让"闭包单元格里的变量"可以赋值修改, 从而兼得"闭包的私密性"与"每函数隔离性"。 - 底层实现: calls 变量存在于 tracer 的帧中, 被 wrapper 以闭包引用 (cell variable) 捕获; nonlocal calls 声明后, calls += 1 修改的就是这个单元格。 - 设计原因: 无全局污染、无类定义, 代码最短; 是"通用函数+方法场景"最推荐的方案。 - 解决什么实际问题: 既要 per-function 计数, 又不想要类那一坨样板——nonlocal 是最优雅的折中。 - 初学者误区: 忘写 nonlocal 就会报 UnboundLocalError——因为 Python 把 calls += 1 视为给局部变量赋值却读到了未定义绑定。这正是"读了没声明"的经典报错。#### 代码分析

最后, 为了处理某些需要变化的状态, 你还可以不用全局与非局部, 改用函数属性 (function attributes) 代替 nonlocal。正如第 17、19 章所见, 用 func.attr = value 的赋值就能给函数挂上任意属性。由于工厂函数每次调用都会造出新函数, 其属性便成了"每次调用一份"的状态。而且, 这个方法只需为"必须变化"的状态使用; 外层作用域引用照旧被保留、正常工作。为了演示, 示例 39-5 只用 wrapper.calls 保存状态, 效果与前面的 nonlocal 版完全相同, 因为计数器依然是"每装饰函数一份"。就像第 17 章学过的, 这之所以行得通, 仅仅因为名字 wrapper 被保留在外层 tracer 函数的作用域中。当我们之后 wrapper.calls += 1 时, 并没有改变名字 wrapper 本身, 所以不需要 nonlocal 声明。这个方案险些被降格为一行脚注——因为它或许比 nonlocal 更晦涩, 也许更适合留到其他方案都不好用的时候用。但函数属性有一个显著优势: 和类实例一样, 它允许从

装饰器代码之外访问保存的状态；`nonlocal` 只能在嵌套函数内部看到，而函数属性可见范围更广。在章末测验的解答中，我们会再次用到函数属性——那时它们对“外部可见”的需求就会派上用场。作为“伴随使用场景的可变状态”，它与 `nonlocal` 等价。老规矩：在多个工具中做选择，本就是编程任务的一部分。因为装饰器常常意味着多级可调用，你可以把“函数 + 外层作用域”“类 + 属性”“函数属性”自由组合成五花八门的结构。不过你很快就会看到，这往往比你想象得微妙——每个被装饰函数应当有自己的状态，而每个被装饰类可能既需要自己的状态，又需要每个生成实例各自的状态。

深度理解

- 核心概念：函数对象本身就是“可携带属性的字典对象”；`wrapper.calls` 把可变状态挂在函数对象上。
- 底层实现：`wrapper` 是函数对象，Python 函数有 `dict`，`wrapper.calls` 实际上是在函数的命名空间字典里加键。
- 设计原因：比 `nonlocal` 更“显式可见”——外部代码可以读 `hack.calls`，而 `nonlocal` 只能在闭包内部访问。
- 解决什么实际问题：需要在被装饰函数之外（如 `DEBUG`、测试脚本）读取/重置状态时，函数或类属性是关键。
- 初学者误区：以为“没有 `nonlocal` 就不能改外层变量”。此处不改名字只改属性，所以不需要 `nonlocal`——区分“重新绑定名字”与“修改属性”是 Python 作用域规则的要点。

代码分析

```
# 39-5decorator_state_attributes.py
def tracer(func):
    def wrapper(*args, **kwargs):
        wrapper.calls += 1      # calls
        print(f'call {wrapper.calls} to {func.__name__}')
        return func(*args, **kwargs)
    wrapper.calls = 0
    return wrapper

@tracer
def hack(a, b, c): ...

@tracer
def code(x, y): ...
```

执行细节：tracer(hack) 先定义 wrapper 函数对象，再执行 wrapper.calls = 0 在函数对象上加属性，然后返回 wrapper 并绑定为 hack。此后 wrapper.calls += 1 直接操作函数对象属性，不涉及名字重新绑定，因此无需 nonlocal。设计思想：以"函数对象的属性"充当状态容器，兼具类的开放性可见与闭包的轻量性。

39.20 Class Pitfall: Decorating Methods (类陷阱：装饰方法)

英文原文

When the preceding section's class-based tracer function decorator, Example 39-2, was initially coded, it was assumed that it could also be applied to any meth

od—decorated methods should work the same, but the automatic self instance argument would simply be included at the front of args. The only real downside to this assumption is that it is completely wrong, though the reasons for the failure are far from obvious. In short, when applied to a class's method, this version of the tracer fails because self is the instance of the decorator class and the instance of the decorated subject class is not included in args at all. This phenomenon was introduced abstractly earlier in this chapter, but now we can see it in the context of working code. Example 39-2's class-based decorator works as advertised for plain functions (copy/pasters: don't copy the initial "... " REPL prompts included in this chapter). However, decoration of class-level methods fails (more lucid sequential readers might recognize this as an adaptation of our Person class resurrected from the object-oriented tutorial in Chapter 28): The root of the problem here is in the self argument of the tracer class's call method—is it a tracer instance or a Person instance? We ultimately need both as it's coded: the tracer for decorator state, and the Person for routing on to the original method. Really, self must be the tracer object to provide access to tracer's state information (its calls and func); this is true whether decorating a simple function or a method. Unfortunately, when our decorated method name is rebound to a class instance object with a call, Python passes only the tracer

instance to self; it doesn't pass along the Person subject in the arguments list at all. Moreover, because the tracer knows nothing about the Person instance we are trying to process with method calls, there's no way to create a bound method with an instance, and thus, no way to correctly dispatch the call. This isn't a bug, but it's wildly subtle. In the end, the prior listing winds up passing too few arguments to the decorated method, and results in an error. Add a line to the decorator's call to print all its arguments to verify this—as you can see, self is the tracer instance, and the Person instance is entirely absent: `<main.tracer object at 0x108a02c00> (0.10,) {}` As mentioned earlier, this happens because Python passes the implied subject instance to self only when a method name is bound to a simple function; when it is an instance of a callable class, that class's instance is passed instead.

中文翻译

前面一节的基于类的 tracer 函数装饰器（示例 39-2）最初编写时，假设它也可以应用到任何方法（method）上——被装饰的方法应该同样工作，隐藏的 self 实例参数会自然出现在 args 的前端。这个假设唯一的“真实缺点”是：它完全错误——不过失败的原因远非显而易见。简言之，当它应用到类的方法上时，这个版本的 tracer 会失败：因为 self 是装饰器类的实例，而被装饰主题类的实例根本没进 args。这个现象在本章前文已经抽象地介绍过，现在我们能具体代码中看到它。示例 39-2 的基于类的装饰器对普通函数

一切正常（复制粘贴党注意：不要连本章 REPL 里开头的 `".."` 提示符一起复制，以免破坏缩进）。但对类级别的方法做装饰却失败了（细心的顺序读者可能会认出，这是第 28 章面向对象教程里 `Person` 类的改编版）：`pat.giveRaise(10)` 会报错 `TypeError: Person.giveRaise() missing 1 required positional argument: 'percent'` 问题根源在 `tracer` 类的 `call` 方法的 `self` 参数上——它到底是 `tracer` 实例还是 `Person` 实例？按现在的写法我们其实两者都要：`tracer` 用来取装饰器状态，`Person` 用来把调用路由到原方法。事实上，`self` 必须是 `tracer` 对象，以提供对 `tracer` 状态信息（`calls` 与 `func`）的访问——无论装饰的是普通函数还是方法都是如此。不幸的是，当我们的被装饰方法名被重绑定为“带 `call` 的类实例”时，Python 只会把 `tracer` 实例传给 `self`；`Person` 主题根本不在参数列表里。而且，因为 `tracer` 对我们要处理的 `Person` 实例一无所知，就没有办法造出一个带实例的绑定方法，因此也就无法正确派发调用。这不是 `bug`，但它极其微妙。最终，前面的清单给被装饰方法传的参数不足，于是报错。往装饰器的 `call` 里加一行打印全部参数即可验证——如你所见，`self` 是 `tracer` 实例，而 `Person` 实例完全缺席：`<main.tracer object at 0x108a02c00> (0.10,) {}` 如前所述，这之所以发生，是因为 Python 只有当方法名绑定到简单函数时，才会把隐藏的主题实例传给 `self`；当方法名绑定到可调用类的实例时，传给 `self` 的取而代之的是那个类自己的实例。

深度理解

·核心概念：“类实例装饰器 $\neq$ 方法装饰器”。`@tracer` 装饰 `giveRaise` 后，`giveRaise` 变成 `tracer` 实例；`pat.give`

Raise(...) 触发的是 `tracer.call(self=tracer实例, args)`, `Person` 实例没有位置可站。

·底层实现：属性解析规则——`pat.giveRaise` 找到的属性是"可调用类实例"时，Python 不做 `bound-method` 包装；只有属性是简单函数时才把实例绑进去。所以 `Person` 永远进不了参数。

·设计原因：这是语言协议的自治结果，非缺陷；理解它必须放弃"self 一定是被装饰对象"的直觉。

·解决什么实际问题：凡是"要同时装饰函数与方法"的通用装饰器，都必须绕开"单个 self 实例兼任两职"的陷阱——嵌套函数方案是正解。

·初学者误区：报错信息里说 `missing 1 required positional argument:'percent'`，初学者常去检查 `Person`，却不知是装饰器的 `call` 参数形态出了问题。

代码分析

```
class Person:
    def __init__(self, name, pay):
        self.name = name
        self.pay = pay

    @tracer
    def giveRaise(self, percent):    # giveRaise = tracer(giveRais...
        self.pay *= (1.0 + percent)

pat = Person('Pat Jones', 50_000)
pat.giveRaise(.10)                # tracer.__call__(tracer...
# → call 1 to giveRaise
# → TypeError: Person.giveRaise() missing 1 required positional ar...
```

调用链: `pat.giveRaise` 属性解析得到 `tracer` 实例 → 调用它 (...) 触发 `call`, 参数只有 `(0.10,)`; `call` 内部 `self.func(args) = giveRaise(0.10)`, 原方法需要 `self` 与 `percent` 两个参数, 于是缺参报错。设计思想: 装饰器若要用可调用类实现, 必须意识到"它接管了方法调用协议", 而方法协议里的 `self` 已被它自己占用。

Using nested functions to decorate methods (用嵌套函数装饰方法)

英文原文

If you want your function decorators to work on both simple functions and class-level methods, the most straightforward solution lies in using one of the other state retention solutions described earlier—code your function decorator as nested `def` statements so that you don't depend on a single `self` instance argument to be both the wrapper class instance and the subject class instance. In fact, we already have—both Examples 39-4 and 39-5 work for both functions and class methods by using nested functions along with nonlocal variables or function attributes. Because decorated methods here are rebound to simple functions instead of instance objects, Python correctly passes the `Person` object as the first argument, and the decorator propagates it on in the first item of `args` to the `self` argument of the real, decorated methods. Trace through t

these results and decorators to make sure you have a handle on this model; the next section provides an alternative to it that supports classes but is also substantially more complex.

中文翻译

如果你希望函数装饰器既能用于普通函数也能用于类级别的方法，最直接的解决方案在于使用前面讲过的其他状态保存方案之一——把函数装饰器写成嵌套的 `def` 语句，从而不再依赖"单个 `self` 实例参数兼任包装类实例与主题类实例两职"。事实上我们早就在这么做了——示例 39-4 与 39-5 通过"嵌套函数 + `nonlocal` 变量或函数属性"，对函数与类方法都能正常工作。因为这里被装饰的方法被重绑定为简单函数而非实例对象，Python 会正确地把 `Person` 对象作为第一个参数传入，装饰器再把它原样放在 `args` 的第一项，传播给真正被装饰方法的 `self`。请亲手追踪一遍这些结果与装饰器，确保你完全掌握了这个模型；下一节将提供一个支持类但复杂得多的替代方案。

深度理解

·核心概念：嵌套函数方案里，`wrapper` 是普通函数，`pat.giveRaise` 是简单函数属性 → Python 会按绑定方法协议把 `pat` 塞进第一个参数 → `wrapper` 的 `args` 第一个元素就是 `Person` 实例。

·底层实现：属性解析找到简单函数 → 创建 `bound method` (绑定 `pat`) → 调用时 `args = (pat, 0.10)` → `wrapper` 转发 `func(args)` 即 `giveRaise(pat, 0.10)`，完美。

·设计原因：把"自我状态"放进闭包而不是 self 字段，是解除"self 冲突"的关键一步——一石二鸟。

·解决什么实际问题：编写"函数与方法通用"装饰器的唯一推荐姿势，后续的 rangetest、计时器修订版都照此办理。

·初学者误区：觉得"反正报错也能看"，不追踪参数流。建议在 wrapper 里打印 args 亲自看一遍 (person实例, 0.1)，这个模型才真正扎根。

代码分析

```
# 39-4decorator_state_nonlocals.py
from decorator_state_nonlocals import tracer

@tracer
def hack(a, b, c):          #
    print(a + b + c)
hack(1, 2, 3)               # call 1 to hack / 6

class Person:              #
    def __init__(self, name, pay):
        self.name = name
        self.pay = pay

    @tracer
    def giveRaise(self, percent): # self args
        self.pay *= (1.0 + percent)

pat = Person('Pat Jones', 50_000)
pat.giveRaise(.10)         # call 1 to giveRaisenonlocal
f'{pat.pay:,.2f}'          # '60,500.00'
```

解释器处理链：@tracer 把 giveRaise 替换为 wrapper
(简单函数) → pat.giveRaise 属性解析：简单函数 →

生成 bound method (含 pat) → 调用时 wrapper(pat, .10) → func(pat, .10) 到达原方法体。设计思想: 让 Python 原生的"绑定方法"机制帮我们传递 self, 装饰器只做透明中转。

Using descriptors to decorate methods (用描述符装饰方法)

英文原文

Although the nested function solution illustrated in the prior section is the most straightforward way to support decorators that apply to both functions and class-level methods, other schemes are possible. The descriptor feature we explored in the prior chapter, for example, can help here as well. Recall from our discussion in the prior chapter that a descriptor is normally a class attribute assigned to an object with a get method run automatically whenever that attribute is referenced and fetched. Descriptors may also have set and del access methods, but we don't need them here. More relevant to this chapter's topic: because the descriptor's get method receives both the descriptor class instance and subject class instance when invoked, it's well suited to decorating methods when we need both the decorator's state and the original class instance for dispatching calls. Consider the alternative racing decorator in Example 39-6, which also happens

s to be a descriptor when used for a class-level method. This works the same as the preceding nested function coding. Its operation varies by usage context:- Decorated functions invoke only its call, and never invoke its get.- Decorated methods invoke its get first to resolve the method name fetch; the object returned by get retains the subject class instance and is then invoked to complete the call expression, thereby triggering the decorator's call. For example, given the same testing code, the call `pat.giveRaise(.10)` runs `tracer.get` first because the `giveRaise` attribute in the `Pers` class has been rebound to a descriptor by the method function decorator. The call expression then triggers the call method of the returned wrapper object, which in turn invokes `tracer.call`. In other words, decorated method calls trigger a five-step process: `tracer.get`, which invokes `wrapper.init`, followed by three call operations—`wrapper.call`, `tracer.call`, and finally the original wrapped method. The wrapper object retains both descriptor and subject instances, so it can route control back to the original decorator/descriptor class instance. In effect, the wrapper object saves the subject class instance available during method attribute fetch and adds it to the later call's arguments list, which is passed to the decorator call.

中文翻译

虽然上一节展示的嵌套函数方案是支持"同时应用于函数与类方法"的最直接方式，但其他方案也是可能的。例如，前一章研究过的描述符（descriptor）特性在这里也能帮上忙。回忆前一章的讨论：描述符通常是"一个带有 get 方法的对象"，被赋给类属性，每当该属性被引用、抓取时 get 就自动运行。描述符也可以有 set 与 del 访问方法，但我们这里用不到。与本章主题更相关的是：由于描述符的 get 方法被调用时，能同时收到描述符类实例与主题类实例，它非常适合用来装饰方法——因为这时我们既需要装饰器状态、又需要原类实例来派发调用。考虑示例 39-6 中的替代跟踪装饰器，它在用于类级别方法时恰好也是一个描述符。它的工作方式与前面的嵌套函数编码相同，操作随使用情境而异：- 装饰函数时只触发它的 call，永远不会触发 get。- 装饰方法时，先触发 get 解析方法名的抓取；get 返回的对象保留了主题类实例，随后该对象被调用来完成调用表达式，从而触发装饰器的 call。例如，给定相同的测试代码，`pat.giveRaise(.10)` 会先运行 `tracer.get`，因为 `Person` 类里的 `giveRaise` 属性已被方法函数装饰器重绑定成一个描述符。调用表达式随后触发返回的 `wrapper` 对象的 call，后者又调用 `tracer.call`。换言之，被装饰方法的调用触发一个五步过程：`tracer.get`（其内部调用 `wrapper.init`），之后是三个调用操作——`wrapper.call`、`tracer.call`，最后是原包装方法。`wrapper` 对象同时保留了描述符实例与主题实例，因此它能把控制权路由回原装饰器/描述符类实例。实际上，`wrapper` 对象保存了"方法属性抓取时可得"的主题类实例，并把它加入后续调用的参数列表，传入装饰器 call。

深度理解

·核心概念：描述符方案的本质——利用 `get(instance, owner)` 一次性同时拿到描述符与主题实例，再用 `wrapper` 把它们粘合成后续调用。

·底层实现：方法被装饰后，`giveRaise` 是 `tracer` 实例（同时也是描述符）；`pat.giveRaise` 触发 `get(tracer实例, Person)`；返回的 `wrapper` 记录 `(desc, subj)`，调用它时构造 `desc(subj, args)` 完成路由。

·设计原因：当"装饰器状态"与"主题实例"都要时，描述符是 Python 提供的唯一"一箭双雕"钩子——比闭包更重，但展示另一种可能。

·解决什么实际问题：提供比嵌套函数更结构化的备选；其代价是复杂度与每方法多步调用的性能损耗，作者明确劝退："如果复杂度没让你崩溃，性能开销也该让你崩溃了"。

·初学者误区：忘了描述符协议只在"类属性"上生效——`@tracer` 装饰普通函数时 `get` 根本不触发，所以"函数走 `call`、方法走 `get+call`"两条路径必须同时维护。

代码分析

```

# 39-6calltracer_desc_class.py
class tracer(object):
    def __init__(self, func):
        self.calls = 0
        self.func = func
    def __call__(self, *args, **kwargs):
        self.calls += 1
        print(f'call {self.calls} to {self.func.__name__}')
        return self.func(*args, **kwargs)
    def __get__(self, instance, owner):
        return wrapper(self, instance)

class wrapper:
    def __init__(self, desc, subj):
        self.desc = desc
        self.subj = subj
    def __call__(self, *args, **kwargs):
        return self.desc(self.subj, *args, **kwargs)

@tracer
def hack(a, b, c): ...

class Person:
    @tracer
    def giveRaise(self, percent): ...

```

五步执行链（方法调用）：pat.giveRaise(.10) → ① tracer.get(pat) → ② wrapper.init(desc=pat实例, subj=pat) → ③ wrapper.call(.10) → ④ tracer.call(pat, .10) → ⑤ 原方法体。设计思想：把"属性抓取"与"调用"两个时刻之间的信息（主题实例）暂存于 wrapper，从而在不依赖绑定方法的情况下正确路由。示例 39-7 用嵌套函数代替 wrapper 类得到等价的更短版本，示例 39-8 则演示了"只有 get 没有 call"的版本——它只能装饰方法、装饰函数就崩（属性访问不会触发 get）。

39.21 Timing Function Calls (为函数调用计时)

英文原文

To better sample what function decorators are capable of, let's turn to a different use case. Our next decorator times calls made to a decorated function—both the time for one call and the total time among all calls. As coded in Example 39-9, the decorator is applied to two functions to compare the speeds of list comprehensions and the map built-in. When run on a macOS host by CPython 3.12, the output of this file's self-test code is as follows—giving for each function call the function name, time for this call, and time for all calls so far, along with the first call's return value, cumulative time for each function, and the map-to-comprehension time ratio at the end. Times vary per Python version, test machine, and other variables, of course, and cumulative time is available as a class instance attribute here. As usual, map calls are slower than list comprehensions when the latter can avoid a function call (or equivalently, require one). For comparison, see Chapter 21 for a nondecorator approach to timing iteration alternatives. In this specific case, a nondecorator approach would allow the subject functions to be used with or without timing, but it would also co

mplicate the call signature when timing is desired—we'd need to add code at every call instead of once at the def. Moreover, there would be no direct way to guarantee that all calls are routed through timing logic. This may make it difficult to collect cumulative data for all calls. In general, decorators may be preferred when functions are already deployed as part of a larger system and may not be easily passed to analysis functions at calls.

中文翻译

为了更充分地采样函数装饰器的能力，我们换个用例。下一个装饰器为被装饰函数的调用计时——既计一次调用的耗时，也计所有调用的总耗时。正如示例 39-9 所写，该装饰器被应用到两个函数上，用来比较列表推导式 (list comprehension) 与 map 内建函数的运行速度。在 macOS 主机上用 CPython 3.12 运行此文件自测代码时，输出如下——对每次函数调用给出函数名、本次调用耗时、迄今所有调用累计耗时，附带首次调用的返回值、每个函数的累计时间与末尾的 map 对列表推导的时间比。listcomp: 0.00000, 0.00000 / listcomp: 0.00366, 0.00366 ... map/comp = 1.408当然，耗时因 Python 版本、测试机器及其他变量而异；这里累计时间以类实例属性的形式可得。和往常一样，当列表推导能避免一次函数调用时，map 比列表推导慢（或者说 map 需要函数调用可能是它慢的原因）。作为对照，第 20 章提供了不用装饰器的计时方法。在这个具体例子中，非装饰器方案可以让这些函数“带或不带计时”两用，但想要计时会搞复杂调用签名——我们必须每个调用点都加

代码，而不是只在 `def` 处加一次。而且也没有直接的办法保证程序里所有调用都经过计时逻辑，这可能让你难以收集全部调用的累计数据。总的来说，当函数已经作为更大系统的一部分部署、不易在调用处传递给分析函数时，人们往往会更偏好装饰器方案。

深度理解

- 核心概念：计时装饰器 = 在 `call` 里"掐秒表、跑函数、再掐表"，累计进 `self.alltime`。定时工具与业务解耦。
- 底层实现：`time.perfcounter()` 取高精度单调时钟；`self.alltime += elapsed` 把每次耗时累加到实例属性中。
- 设计原因：把基准测试（benchmark）变成"声明式"工具——用 `@timer` 装饰即可对比两种写法的快慢，不改任何业务代码。
- 解决问题：性能对比、热点定位；`allTime` 作为属性可被外部读取，`map/comp` 比率直观量化差距。
- 初学者误区：用装饰器做精细基准时忘了它本身有包装开销；书里特意用第 21 章非装饰器方案（`timer`、`timeit`）对照，提醒两种手段并存互补。

代码分析


```

# 39-9timerdeco1.py
"Caveat: timer won't work on methods as coded (see quiz solution)"
import time, sys
class timer:
    def __init__(self, func):
        self.func = func
        self.alltime = 0
    def __call__(self, *args, **kargs):
        start = time.perf_counter()
        result = self.func(*args, **kargs)
        elapsed = time.perf_counter() - start
        self.alltime += elapsed
        print(f'{self.func.__name__}: {elapsed:.5f}, {self.alltime...}')
        return result

@timer
def listcomp(N):
    return [x * 2 for x in range(N)]

@timer
def mapcall(N):
    return list(map((lambda x: x * 2), range(N)))

if __name__ == '__main__':
    for func in (listcomp, mapcall):
        result = func(5)
        func(50_000); func(500_000); func(1_000_000)
        print(result)
        print(f'allTime = {func.alltime}\n')
    print('**map/comp = ', round(mapcall.alltime / listcomp.alltime...))

```

解释器执行链：@timer 时 timer.init 建实例；每次 listcomp(...) → timer.call → 计时并转发。func.alltime 以实例属性暴露累计耗时。设计思想：计时、对比、累计，全部内聚到装饰器；业务函数保持纯净。注意本版是"类装饰器"，不能直接用类方法——提示语已写"quiz solution"

39.22 Adding Decorator Arguments (添加装饰器参数)

英文原文

The timer decorator of the prior section works, but it would be nice if it were more configurable—providing an output label and turning trace messages on and off, for instance, might be useful in a general-purpose tool. Decorator arguments come in handy here: when they're coded properly, we can use them to specify configuration options that can vary for each decorated function. A label, for instance, might be added as abstractly follows. This code adds an enclosing scope to retain a decorator argument for use on a later actual call. The structure of a decorator-factory is:

```
def timer(label=""): -> def decorator(func): -> def onCall(args): -> func(args); print(label,...)
```

When the listcomp function is defined, Python really invokes decorator—the result of timer, run before decoration actually occurs—with the label value available in its enclosing scope. That is, timer returns the decorator, which remembers the decorator argument and the original function, and returns the callable onCall, which ultimately stops the original function on later calls. Because this structure creates new decorator and onCall funct

ions, their enclosing scopes are per-decoration state retention. We can put this structure to use in our timer to allow a label and a range computer flag to be passed in at decoration time. Example 39-10 does just that, coded in a module file so it can be imported as a general tool; it uses a class for the second state retention level.

中文翻译

上一节的计时装饰器能工作，但要是它能更可配置就更好了——比如提供输出标签 (label)、开关跟踪消息，这对通用工具可能很关键。此刻装饰器参数就派上用场了：只要编码得当，我们就可以用它们来指定随每个被装饰函数而变化的配置选项。比如，标签可以如下抽象地添加。这段代码新增一层外层作用域，用来把装饰器参数保留给后续实际调用使用。装饰器工厂的结构是：外层 `def timer(label="")` → 中层 `def decorator(func)` → 内层 `def onCall(args)` (内部 `func(args)` 且打印 `label` 等) → 依次返回。当 `listcomp` 被定义时，Python 实际上是在装饰尚未发生之前就调用 `timer` (其结果 `decorator` 以标签作为参数)，外层已经携带着 `label` 的值。也就是说，`timer` 返回装饰器；装饰器记住装饰器参数与原函数，返回可调用 `onCall`；`onCall` 才是以后调用时执行的。由于这个结构创建了新的 `decorator` 与 `onCall` 函数，它们的外层作用域就是“每装饰一份”的状态保存。我们可以把这个结构用进计时器，让“标签”和“跟踪开关”在装饰时刻传入。示例 39-10 正是这样做的——它把计时器编成模块文件，便于作为通用工具被 `import`；它用类（而非嵌套函数）来实现第二层状态保存，但净效果相

似。

深度理解

·核心概念：装饰器工厂三级结构：timer(label)（读配置）→ Timer/decorator（接收函数）→ call/onCall（处理调用）。

·底层实现：label、trace 存活在 timer 函数帧的闭包里；Timer 类实例在 @ 时刻创建并保存 self.func。

·设计原因：配置参数必须在"装饰时刻"固定，不能每次调用再变；外层闭包正是为长期保存配置而生。

·实际问题：同一套计时器在不同函数上输出不同标签（[CCC]==> 与 [MMM]==>），大程序输出里能一眼分辨；trace=False 可无人值守关闭。

·初学者误区：把 @timer(label='x') 当作"调用 timer 后再装饰"的两步，理解对了三步的一半——务必看到 timer (...) 已经先执行了（参数已解析完）。

代码分析

```
# 39-10timerdeco2.py
import time
def timer(label='', trace=True):    #
    class Timer:
        def __init__(self, func):    # @
            self.func = func
            self.alltime = 0
        def __call__(self, *args, **kargs):    #
            start = time.perf_counter()
            result = self.func(*args, **kargs)
            elapsed = time.perf_counter() - start
```

```

        self.alltime += elapsed
    if trace:
        # /
        if label: print(label, end=' ')
        print(f'{self.func.__name__}: {elapsed:.5f}', {self...
    return result
return Timer

# 39-11testseqs.py
from timerdeco2 import timer
@timer(label='[CCC]==>')
def listcomp(N):
    # listcomp = timer(...)(listco...
    return [x * 2 for x in range(N)]

@timer(trace=True, label='[MMM]==>')
def mapcall(N):
    return list(map((lambda x: x * 2), range(N)))

```

执行时序：@timer(label='[CCC]==>') 求值时：① timer('[CCC]==>') 立即返回 Timer 类 → ② 类 body 被装饰 Timer(listcomp) 建实例、保存 func、alltime=0 → ③ listcomp(...) 调用 Timer.call。注意：单实例保存于"第二层类的实例"，而标签保存在第一层闭包中。设计思想：用"每层一个生命周期"隔开配置、函数、调用三类信息，让同一装饰器能以不同标签复用。（书中还强调：此计时器仍不能用于类方法——这正留作章末测验第 1 题。）

39.23 Coding Class Decorators (编写类装饰器)

英文原文

So far, we've been coding function decorators to manage function calls, but as we've seen, decorators work on classes too. As described earlier, while similar in concept to function decorators, class decorators are applied to classes instead—they may be used either to manage classes themselves or to intercept instance-creation calls in order to manage instances. Also like function decorators, class decorators are configurable and syntactic sugar for an explicit rebinding step.

中文翻译

到目前为止，我们一直在编写函数装饰器去管理函数调用，但正如我们所见，装饰器对类同样适用。如前述，虽然概念上与函数装饰器类似，类装饰器是被应用到类上的——它们既可以用来管理类本身，也可以用来拦截"实例创建调用"以管理实例。与函数装饰器一样，类装饰器也是"幕后手工重绑定的显式糖"。

深度理解

- 核心概念：类装饰器两大用途——"管理类本身"与"拦截实例创建以管理实例"，本节起依次演示。
- 底层实现：`C = decorator(C)`；此后 `C()` 进入装饰器返回的包装器。
- 设计原因：用 `@` 把"对每个创建出来的实例做统一处理"写进类定义本身，代码即声明。

·实际问题：单例、对象接口代理、ORM 表绑定——"凡是围绕一个类的约定"，装饰器都是清爽的表达。

·初学者误区：把"装饰类的类装饰器"与"返回包装类的工厂函数"搞混；前者每次 C() 需小心共享状态（下节的"多实例陷阱"）。

39.24 Singleton Classes (单例类)

英文原文

Let's start with something simple. By intercepting instance-creation calls, class decorators can be used to either manage all the instances of a class, or augment the interfaces of those instances. Example 39-12 lists a first class decorator example that does the former—managing all instances of a class. This code implements the classic singleton coding pattern, where at most one instance of a class ever exists. Its singleton function defines and returns the function, and the @ syntax automatically wraps up a subject class in this function. To use this, decorate the classes for which you'd like to enforce a single-instance model, as in Example 39-13. Now, when the Person or Hack class is later used to create an instance, the wrapping logic layer provided by the decorator routes instance-creation calls to onCall, which in turn ensures a single instance per class, regardless of how many construction calls are made. Here's output: Sue 1000 / Sue 1000 / 42

中文翻译

让我们从简单例子开始。通过拦截实例创建调用，类装饰器既可以用来管理"一个类的所有实例"，也可以用来增强这些实例的接口。示例 39-12 给出的是前者——管理某类的所有实例。这段代码实现了经典的单例（singleton）编码模式：一个类最多只存在一个实例。singleton 函数定义并返回 onCall，@ 语法自动把主题类包装进这个函数。要使用它，就对想实施"单实例模型"的类加上装饰，如示例 39-13 所示。装饰 Person 与 Hack 两个类。现在，当 Person 或 Hack 类之后被用来创建实例时，装饰器提供的包装逻辑层会把实例创建调用路由给 onCall，onCall 保证"每类只有一个实例"——不管创建调用发生多少次。本书随之给出运行输出：Sue 1000 / Sue 1000 / 42 42。（解读：第二次 Person('Bob',...) 实际返回的还是 Sue 那个对象，bob.name 仍打印 Sue；X 与 Y 两个 Hack 实例也是同一个，故两个 attr 都是 42。）

深度理解

- 核心概念：单例模式 = "每个类只允许一个实例"。类装饰器通过"拦截创建调用 → 查表/复用"实现。
- 底层实现：instances 全局字典以"类对象"为键、以实例为值；onCall 每次创建时先查 if aClass not in instances，存在则直接返回旧实例。
- 设计原因：类装饰器版的单例把"曾否创建"的表格逻辑从类本身拿出，附在装饰层，类保持纯粹。

·最实用的动机：配置中心、连接池、全局状态存储——"
全程序唯一实例"需求广泛。

·初学者误区：以为 Person、Hack 各实例是不同的。实际上 bob 与 sue 指向同一个对象（都打印 Sue 1000）——这是验证单例最直观的方式。也注意用全局字典（每个类一个条目）与用 nonlocal（每类一个作用域）两种实现殊途同归。

代码分析

```
# 39-12singletons.py
instances = {}                                # →
def singleton(aClass):                        # @
    def onCall(*args, **kwargs):            #
        if aClass not in instances:        #
            instances[aClass] = aClass(*args, **kwargs) # ...
        return instances[aClass]           #
    return onCall
# 39-13
@singleton      # Person = singleton(Person)Person onCall
class Person:
    def __init__(self, name, hours, rate): ...
sue = Person('Sue', 50, 20)      # onCall('Sue', 50, 20)
bob = Person('Bob', 40, 10)     # onCall— sue
print(sue.name, sue.pay())      # Sue 1000
print(bob.name, bob.pay())      # Sue 1000 ← sue
```

执行细节：Person 经 @singleton 重绑定为各 onCall；两次实例化都走进 onCall；首次将真实例存入 instances [Person]，第二次直接返回同一对象。设计思想：把"唯一性保证"从类内部移到装饰层——类保持普通，句法外统一管控。非 local 版用每类一个 instance=None 的闭包变量代替全局字典，二者等价。

Singleton coding alternatives (单例的其他实现)

英文原文

Interestingly, you can also code a more self-contained solution where the enclosing-scope changes via `nonlocal`. The following alternative achieves an identical effect, by using one enclosing scope per class, instead of one global table entry per class. You can also code a self-contained solution with either function attributes or a class instead. The first option leverages the fact that there is one `onCall` function per decoration — the function object's namespace serves the same role as an enclosing scope. The second uses one instance per decoration, relying on the same coding pattern that we'll later label a common decorator class pitfall — here we want just one instance, but that's not often the case.

中文翻译

有意思的是，你也可以用 `nonlocal` 修改外层作用域名的方式来写一个更自包含的版本。下面的替代方案效果完全相同：每个类用一个外层作用域，而不是每个类一个全局表条目。它不依赖装饰器之外的全局作用域里的名字（这里 `None` 检查也可用 `==` 或 `is`，都无所谓）。此外，你还可以用函数属性或类实现同样自包含的版本。第一个法子利用“每次装饰产生一个 `onCall` 函数”这个事实——函数对象的命名

空间可以充当外层作用域；第二个法子每装饰用一个实例——它依赖的正是我们稍后要称之为“装饰器类陷阱”的那种编码模式：这里我们恰好想要一个实例，但通常我们并不想要

。

深度理解

·核心概念：单例的三种自包含变体：nonlocal 闭包变量、函数对象属性、装饰器类属性——状态容器的选择决定实现形态。

·底层实现：nonlocal 版把 instance 藏进闭包；函数属性版把 instance 挂在 onCall 函数对象上；类版存在 self.instance。

·设计原因：体现“多工具殊途同归”——作者有意展示同一种行为有多种编码姿态，训练读者在状态容器间自如切换

。

·实际问题：模块化封装时，尽量“自包含”（不依赖外部全局字典），便于复用、移植。

·初学者误区（重要预告）：类版单例“刻意”用了一个类是“单实例”的巧合工作；同样代码换到别处（如 Tracer 接口跟踪）就会因“每类一个实例、每实例只有一个 wrapper”而翻车。这正是“类陷阱”的前奏。

代码分析

```

# 1 nonlocal
def singleton(aClass):          # @
    instance = None
    def onCall(*args, **kwargs): #
        nonlocal instance
        if instance == None:
            instance = aClass(*args, **kwargs) #
        return instance
    return onCall

# 2 onCall
def singleton(aClass):
    def onCall(*args, **kwargs):
        if onCall.instance == None:
            onCall.instance = aClass(*args, **kwargs)
        return onCall.instance
    onCall.instance = None
    return onCall

# 3 self.instance
class singleton:
    def __init__(self, aClass):    # @
        self.aClass = aClass
        self.instance = None
    def __call__(self, *args, **kwargs): #
        if self.instance == None:
            self.instance = self.aClass(*args, **kwargs)
        return self.instance

```

三种方案行为相同。设计思想（作者的建议）：挑一版写进 importable 模块即可；类版"显式、有结构"，便于将来演化，但 OOP 不必处处用。

39.25 Tracing Object Interfaces (跟踪对象接口)

英文原文

The singleton example of the prior section illustrated using class decorators to manage all the instances of a class. Another common use case for class decorators augments the interface of each generated instance. Class decorators can essentially install a wrapper or "proxy" logic layer atop instances that manages access to their interfaces. For example, in Chapter 31, the `getattr` operator-overloading method was shown as a way to wrap up entire object interfaces of embedded instances in order to implement the delegation coding pattern. Recall that `getattr` is run when an undefined attribute name is fetched; we can use this hook to intercept method calls in a controller class and propagate them to an embedded object.

中文翻译

上一节的单例示例演示了用类装饰器管理"一个类的所有实例"。类装饰器另一常见用法是增强每个生成实例的接口。从根本上说，类装饰器可以在实例之上安装一层包装器或"代理 (proxy)"逻辑层，拦截对实例属性的访问。比如第 31 章曾展示，`getattr` 运算符重载方法可以包裹"被嵌入实例的整个对象"接口，从而实现委托编码模式。回忆一下：一个未定义的属性名被抓取时，`getattr` 会运行；我们可以用

这个钩子在控制器类里拦截方法调用，并把它们转发给被嵌入的对象。

深度理解

- 核心概念：两种"全量管理"的差异——单例管"类的所有实例"，接口跟踪管"每个实例的全部接口"。
- 底层实现：Wrapper.getattr 拦截一切未定义属性访问；已定义的（Wrapper 自己的）属性不受影响，因此 fetches 等计数属性不被跟踪。
- 设计原因：委托模式的价值——不需要知道被包装对象的完整接口，getattr + getattr 就能"原样转发"。
- 实际问题：调试时观察"外部到底碰了哪些属性"，或做访问审计、权限前置。
- 初学者误区：以为 getattr 能拦截一切属性。它只拦截"未定义"属性；内置运算（如 +、[]）的隐式访问它接不住——这个洞尾段专门修补。

The nondecorator approach（非装饰器方案）

英文原文

```
For reference and review, here's the original nondecorator delegation example: class Wrapper: def init(self, object): self.wrapped = object # Save object def getattr(self, attrname): print('Trace:', attrname) # Trace fetch return getattr(self.wrapped, attrname) # Delegate fetch x = Wrapper([1,2,3]) # Wrap a list object x.appe
```

nd(4) # Delegate to list method In this code, the Wrapper class intercepts access to the wrapped object's explicitly named attributes, prints a trace message, and uses the getattr built-in to pass off the request to the wrapped object. This whole-interface model differs from the behavior of function decorators, which wrap up one specific method.

中文翻译

作为参考与复习，这里给出最初的非装饰器委托示例：

```
class Wrapper:
    def init(self, object):
        self.wrapped = object
    # 保存对象
    def getattr(self, attrname):
        print('Trace', attrname)
        # 打印跟踪信息
        return getattr(self.wrapped, attrname)
        # 委托属性

x = Wrapper([1,2,3])
# 包装一个列表对象
x.append(4)
# 委托给列表方法

在这段代码里，Wrapper 类拦截对被包装对象的显式属性的访问，打印一条跟踪消息，并用 getattr 内建函数把请求转给被包装对象。这个 "整体界面 (whole-interface)" 模型与函数装饰器（只包装一个具体方法）的行为不同。
```

深度理解

- 核心概念：委托 (delegation) 模式的三件套：构造时嵌入对象、getattr 拦截、getattr 转发。
- 底层实现：x.append 里 append 不是 Wrapper 实例的属性 → 落入 getattr → 打印、转发。已定义属性（如 wrapped）不触发。
- 设计原因：委托是不知接口细节时的万能转发——不做方

法清单，只管“你有就转”。

·实际问题：跟踪“外部对对象属性的每次触碰”，为接口监控工具奠基。

·初学者误区：委托后还指望 x 是 list 类型；isinstance(x, list) 为 False——类型不再是原类型，这是代理的固有代价。

代码分析

```
class Wrapper:
    def __init__(self, object):
        self.wrapped = object      #
    def __getattr__(self, attrname):
        print('Trace:', attrname)  #
        return getattr(self.wrapped, attrname)  #

x = Wrapper([1, 2, 3])            #
x.append(4)                        # Trace: append [1,2,3,4]
```

执行链：x.append 属性查找，Wrapper 类无该方法、实例 dict 也没有 → 落入 getattr('append') → 打印、getattr(被包装列表, 'append') 取出真实方法 → 调用它（即 [1,2,3].append(4)）。设计思想：属性钩子 + 内建转发 = 极简但完整的接口代理。

39.26 The class-decorator approach（类装饰器方案）

英文原文

Class decorators provide an alternative and convenient way to code this `getattr` technique and wrap an entire interface. The preceding code, for example, can be coded as a class decorator that triggers wrapped instance creation instead of passing premade instances into the wrapper's constructor. Example 39-14 codes this `mod`, supports keyword arguments with `kargs`, and counts the number of accesses to illustrate changeable state. It is important to note that this is very different from the tracer decorator we met earlier (despite the name!) earlier. There, decorators traced and timed calls to a given function or method. By intercepting instance-creation calls, the class decorator here allows us to trace an entire object interface—accesses to any of the instance's attributes. It's also important to note that this decorator's `getattr` won't catch the implicit attribute fetches of built-in operations per the prior chapter, but we'll defer more on this until we code attribute privacy ahead.

中文翻译

类装饰器提供了一种替代且便捷的方式来编码这种 `getattr` 技术并包装整个接口。例如，上面的代码可以改写成“类装饰器触发被包装实例的创建”而不是“把预制实例传进包装器构造函数”。示例 39-14 做了这种改动，用 `kargs` 支持关键字参数，并计数访问次数以演示可变状态。必须注意：这与

我们早先遇到的跟踪装饰器（尽管同名！）完全不同。在那里，装饰器跟踪的是"对给定函数或方法的调用"。而这里，通过拦截实例创建调用，这个类装饰器让我们能跟踪整个对象接口——即对实例任何属性的访问。还需要注意：这个装饰器的 `getattr` 捕获不了内置运算隐式触发的属性抓取（照上一章所述），这个问题我们要到实现属性隐私时再详细讨论。

深度理解

- 核心概念："接口跟踪" ≠ "调用跟踪"——前者管"实例上的每次属性访问"，后者管"某函数被调用了几次"。
- 底层实现：`Tracer(aClass)` 工厂每次创建返回内部 `Wrapper` 类；每次 `C()` 实例化都新建一个 `Wrapper` 实例，`self.wrapped` 与 `self.fetches` 各自独立。
- 设计原因：把"包装接口"的需求推到装饰层，客户端照常写 `Person('Bob', 40, 50)`，一切管理外挂自动生效。
- 实际问题：对每个实例的每属性访问做跟踪、校验、计数的通用基础设施——这正是后面"属性隐私"装饰器的直接前身。
- 初学者误区：误以为 `work.fetches` 也会被跟踪；不会——`fetches` 是 `Wrapper` 自身已定义属性，`getattr` 只拦截未定义的。

代码分析

```

# 39-14interfacetracer.py
def Tracer(aClass):          # @
    class Wrapper:
        def __init__(self, *args, **kargs):    #
            self.fetches = 0
            self.wrapped = aClass(*args, **kargs)    #
        def __getattr__(self, attrname):        #
            print('Trace:', attrname)
            self.fetches += 1
            return getattr(self.wrapped, attrname)    #
    return Wrapper

if __name__ == '__main__':
    @Tracer
    class Hack:
        def display(self): print('Hack!' * 3)
    @Tracer
    class Person:
        def __init__(self, name, hours, rate): ...
        def pay(self):
            return self.hours * self.rate    #

work = Hack(); work.display()    # __getattr__ 'Hack!Hac...
print([work.fetches])            # [1]
bob = Person('Bob', 40, 50); print(bob.name)    # Trace: name →...
print(bob.pay())                  # 2000
sue = Person('Sue', rate=100, hours=60)    # Wrapper Person
print('calls:', [bob.fetches, sue.fetches])    # calls: [4, 2]

```

执行链：@Tracer 使 Person 指向 Wrapper 类；bob = Person('Bob',...) → Wrapper.init 内 aClass(...) 构造真 Person，嵌入 self.wrapped；之后 bob.name 触发 getattr。sue 是另一个独立的 Wrapper 实例，故 fetches 计数互不干扰。设计思想：工厂每次返回“新 Wrapper 类”，每实例 Wrapper 各带状态——为多实例安全与每类作

用域的双保障。

Applying class decorators to built-in types (应用于内置类型)

英文原文

Also note that the preceding one decorates a user-defined class. Just like in the original example in Chapter 31, we could also use the decorator to wrap up built-in objects such as lists, as long as we either subclass to allow decoration syntax or perform the decoration rebinding manually—decorator syntax requires a class statement for the `@` line. As displayed, there are two ways to wrap a built-in. The decorator approach allows us to move instance creation into the decorator itself instead of a premade object. Although this makes little difference in these simple demos, in general it lets us retain normal instance-creation syntax and limits augmentation syntax to class definition. Rather than requiring all instance-creation calls to route objects through a wrapper manually, we need only augment class definitions with decorator syntax. Assuming you will make more than one instance of a class and want to apply the augmentation to every instance of a class, decorators will generally be a net win in terms of both code size and code maintenance.

中文翻译

还需要注意：上文装饰的是用户自定义类。不过正如第 31 章最初示例那样，我们也可以让这个装饰器去包裹内置对象类型（如 list）——只要要么用子类配合装饰语法（@ 行必须配 class 语句），要么手工执行装饰重绑定。有两种方式包裹内置类型。装饰器方案让我们把实例创建移进装饰器本身，而不是要求传入预制对象。尽管这在简单演示里差别不大，但一般来说它让我们保留正常的实例创建语法，并把增强语法限制在类定义处。这不需要让所有实例创建调用都手工把对象塞进包装器，而只需在类定义上用装饰语法。假设你要为一个类创建多个实例，并希望增强应用到该类的每个实例，那么从代码量与代码维护两个角度来看，装饰器通常都是净胜的。

深度理解

- 核心概念：给 list 等内置类型加外衣，有"子类 + @Tracer"与"手工重绑定 MyList = Tracer(list)"两条路。
- 底层实现：class MyList(list): pass 让 @Tracer 语法可落在 class 行；MyList = Tracer(list) 则绕过语法约束直接调工厂。
- 设计原因：装饰器把"创建与包装"合并成一步，调用方仍写 MyList([1,2,3])——零负担。
- 实际问题：对任何（含第三方内置）类加接口管理，通常只需加一行 @，不必改写实例创建处。
- 初学者误区：对内置类型强行写 @Tracer 到没有 class 语句的地方；正确姿势要么子类化，要么手工重绑定。

代码分析

```
from interfacetracer import Tracer

@Tracer
class MyList(list): pass      # MyList = Tracer(MyList)
x = MyList([1, 2, 3])        # Wrapper()
x.append(4)                  # __getattr__ → list.append
x.wrapped                    # [1, 2, 3, 4]

MyList = Tracer(list)        #
x = MyList([4, 5, 6])
x.append(7)                  # Trace: append → [4,5,6,7]
```

对比非装饰器写法 `bob = Wrapper(Person('Bob', 40, 50))`——装饰式 `bob = Person(...)` 更干净。设计思想：把“对象构造过程”纳入装饰层管理，外部代码零变化。

39.27 Class Pitfall: Retaining Multiple Instances (类陷阱：保留多个实例)

英文原文

Curiously, the decorator function can be coded instead, almost, as a class with `set`. Example 39-15 alternative is coded as a class with `init`, `call`, and `getattr` to provide identical logic to the previous version—the `init` triggers on `@` decorator, the `call` on instance creation, `getattr` on attribute fetch. As we saw in the abstract earlier, though, this class-only alternative handles multiple classes as before, but it won't quite work for mul

multiple instances of a given class: each instance-creation call triggers `call`, which overwrites the previous instance. The net effect is that the tracer saves just one—the last wrapped. Example 39-16 extends this file to demo the problem. This code's output follows—the second overwrites the first. The problem here is bad state retention—we make one decorator instance per class but not per class instance, such that only the last instance is retained. The solution, as in our prior class pitfall for decorating methods, lies in abandoning an example adapted to a class-based decorator. The earlier function-based Tracer version does work for multiple instances. Because it returns a class instead of an instance of that class, each instance-creation makes a new Wrapper instance instead of overwriting the state of a single shared Tracer instance. The moral here: decorators are not only arguably magical, they can also be incredibly subtle!

中文翻译

有趣的是，前一个例子的装饰器本身几乎也可以“用封装成类”的方式实现——用恰当的运算符重载协议。示例 39-15 的替代编码就把 Tracer 写成一个类：@ 装饰类时触发它的 `init`，创建主题类实例时触发 `call`，访问属性时触发 `getattr`。正如我们前面抽象地讨论过的，这种纯类替代能像以前一样处理多个类，但对“同一个类的多个实例”却有问题：每次实例创建调用都会触发 `call`，而 `call` 覆盖了上一个实例。净效果是 Tracer 只保存一个——最后创建的那个。示例 3

9-16 把这段扩展一下以演示问题。其输出结果如下：第二个覆盖第一个——bob 名字打印出 Sue！问题在于糟糕的状态保存：我们每类造了一个装饰器实例，但没有“每实例”一个，结果只有最后一个被保留。解决方案与我们前面处理“方法装饰类陷阱”时一样——放弃基于类的装饰器。前面基于函数的 Tracer（示例 39-14）是能处理多实例的。因为它返回的是一个类（Wrapper），而非那个类的实例，每次实例创建都会新建 Wrapper 实例，而不是覆盖一个共享 Tracer 实例的状态。结论：装饰器不光可以说得神乎其神——它们还能极其微妙地出错！

深度理解

- 核心概念：同一个 Tracer 类方案，作为“类装饰器”会在多实例场景翻车——状态粒度错了。
- 底层机制：call 的 `self.wrapped = self.aClass(...)` 每次都写同一个 `self.wrapped` 属性；第二个 `Person('Sue')` 直接把第一个 `Person('Bob')` 顶掉了。
- 设计原因：这是“状态该放在哪个对象上”的经典对照：函数版返回类→状态的载体是每个实例；类版返回实例→状态全堆在同一个装饰器实例上。
- 实际问题：任何“要为某类包装多实例”的装饰器，都不要把 `per-instance` 状态放在装饰器实例身上。
- 初学者误区：这种 bug 的先导症状是：明明实例化两次，却返回（打印）同一个值。看到了这个现象，就该想到“每实例状态被覆盖”。

代码分析

```
# 39-15/16interfacetrapper-fail.py
class Tracer:
    def __init__(self, aClass):          # @
        self.aClass = aClass           #
    def __call__(self, *args):          #
        self.wrapped = self.aClass(*args) # ONELASTINSTANCE PE...
        return self
    def __getattr__(self, attrname):
        print('Trace:', attrname)
        return getattr(self.wrapped, attrname)

@Tracer                                # __init__
class Hack:
    def display(self): print('Hack!' * 3)
work = Hack(); work.display()          # __call__ __getattr__

@Tracer
class Person:
    def __init__(self, name): self.name = name

bob = Person('Bob'); print(bob.name)    # Trace: name → Bob
sue = Person('Sue'); print(sue.name)    # Trace: name → Sue
print(bob.name)                        # OOPS: bob 'Sue'
```

执行细节：每次 `Person('...')` 都调用同一个 `Tracer` 实例的 `call`（同一 `self`），`self.wrapped` 被覆盖；`bob` 与 `sue` 其实都是同一个 `Tracer` 实例（`self` 相同），故 `bob.name` 一变都被改写。对照：函数版返回 `Wrapper` 类，于是 `Person()` 每次生成不同的 `Wrapper` 实例，`wrapped` 与 `fetches` 各自独立。

39.28 Example: "Private" and "Public" Attributes (案例一: "私有"与"公共"属性)

英文原文

The final two sections of this chapter present larger examples of decorator use, which give us a chance to see how concepts come together in more useful code. Both are presented with minimal description, partly to conserve space but mostly because you should already understand decorator basics well enough.

中文翻译

本章最后两节呈现两个更大的装饰器应用案例，让我们有机会看到各个概念如何汇聚进更有用的代码。两者都以极简的叙述呈现——部分是为了节省篇幅，但主要因为你对装饰器基础应该已理解得足够好，完全可以自行研读。

深度理解

- 核心概念：两大收官案例，"属性隐私"与"参数范围校验"，是前文所有技巧（嵌套、代理、状态保存、内省）的集大成练习。
- 底层实现：两者靠的都是 `getattr/setattr` 拦截 + 委托，以及函数内省（code）匹配参数名。
- 设计原因：Lutz 故意少讲解、多读代码——这正是锻炼"读别人代码"能力的刻意安排。

·实际问题：开发期"防范误用"、生产期"去掉开销"；两案例工具化后可以导入复用。

·初学者困境：案例天然"硬"。建议对照运行输出逐步追踪——先理解四层作用域/状态，再读外层包装。

39.29 Implementing Private Attributes (实现私有属性)

英文原文

First up, the class decorator in Example 39-17 implements a Private declaration and access checks for class instance attributes—attributes stored on each instance, or inherited from its classes. This decorator disallows fetch and change access to the attributes from outside the decorated class but still allows the class to access those names within its own methods. It's not quite the same as "private" in C++ or Java, but it provides similar access validations as an option for cases where this might be useful during development. We saw an initial and incomplete implementation for instance attribute privacy in Chapter 30; this version extends it to validate fetches, too, and it uses delegation instead of inheritance to implement the model. Although it implements attribute privacy through the sugar of class decorators, attribute interception is still based upon the `getattr` and `setattr` operator-overloading methods we met in prior chapters. When a private ac

cess is detected, it raises an exception. Example 39-17 lists the decorator's first-cut code, along with a self-test. As coded, it catches all explicit attribute fetches, but not the implicit fetches of built-in operations.

中文翻译

首先登场的是类装饰器（示例 39-17），它实现了带访问校验的 `Private` 声明：操作用于"存储在实例上、或从所属类继承来的类实例属性"。该装饰器禁止从被装饰类的外部对这些属性做抓取与修改访问，但允许类自身在自己的方法中自由使用这些名字。它并不完全等于 C++ 或 Java 的 "`private`"，但提供类似的访问校验，作为开发期特定场景下的一种可选方案。我们在第 30 章见过一个针对"只改不改读"的实例属性隐私的初始且不完整实现；本示例把它扩展到抓取也要校验，并且用委托（delegation）而非继承来实现模型。尽管它是借助类装饰器语法糖来实现属性隐私的，其属性拦截最终还是基于 `getattr` 与 `setattr` 运算符重载方法。一旦检测到私有属性访问，就引发一个带错误消息的异常，可用 `try` 捕获，也可任其终止脚本。示例 39-17 列出装饰器的初稿代码与模块底部的自测。如所写，它能守住显式属性抓取，但抓不住内置运算的隐式抓取（稍后详谈）。

深度理解

- 核心概念：私有属性 = 用代理包装类实例 + "名单制"访问校验：privates 列表里的名字禁止外部 `get/set`。
- 底层实现：`Private(privates)` 工厂 → `onDecorator(aClass)` → `onInstance (proxy 类)`；`getattr` 里 `attr in privates` 则 `raise TypeError`，否则委托；`setattr` 里对 `wra`

pped 特殊放行、对非私有名转发到 wrapped。

·设计原因：用"白名单"式模拟 OO 语言的访问控制；Python 哲学是"人人自觉"，Lutz 明说"sparingly（少用）"

。

·解决问题：开发期防止误改关键属性；异常把违规访问变成立即暴露的崩溃，胜过静默错误。

·初学者误区：以为拦截了"类内"访问——委托模式只拦截外部访问，类内方法里 self.data 走正常路径、零校验（隐私不继承）。

·提示：trace 辅助函数用 3.12 的嵌套 f-string (f'[{...}']') 打印调试。

代码分析

```
# 39-17access1.py·
traceMe = False
def trace(*args):
    if traceMe: print(f'[{ ' '.join(map(str, args))}]') # Python ...

def Private(*privates):
    # privates
    def onDecorator(aClass):
        # aClass
        class onInstance:
            #
            def __init__(self, *args, **kargs):
                self.wrapped = aClass(*args, **kargs)
            def __getattr__(self, attr): # getattr
                trace('get:', attr) # wrapped
                if attr in privates:
                    raise TypeError('private attribute fetch, ' + ...
                else:
                    return getattr(self.wrapped, attr)
            def __setattr__(self, attr, value): #
                trace('set:', attr, value) #
```

```

        if attr == 'wrapped':
            #
            self.__dict__[attr] = value
        elif attr in privates:
            raise TypeError('private attribute change, ' + ...
        else:
            setattr(self.wrapped, attr, value)
    #
    return onInstance
return onDecorator

if __name__ == '__main__':
    traceMe = True
    @Private('data', 'size')
    # Doubler = Private(...)
    class Doubler:
        def __init__(self, label, start):
            self.label = label
            self.data = start
        def size(self):
            return len(self.data)
        def double(self):
            for i in range(self.size()):
                self.data[i] = self.data[i] * 2
        def display(self):
            print(f'{self.label} => {self.data}')

```

执行链：@Private('data','size') → 先 Private('data','size') 返回 onDecorator → 调 onDecorator(Doubler) 返回 onInstance 类并赋给 Doubler。之后每次 Doubler(...) 创建 onInstance 代理，内部嵌入真实例。外部访问 X.label 合法转发；X.data 或 X.size 因在 privates 名单中被 raise 拦截。设计要点：setattr 设置自己 wrapped 时必须走 self.dict[attr]=value 绕开递归，这一反直觉技巧正是阅读关键。

39.30 Implementation Details (实现细节)

英文原文

This code is nontrivial, and you're probably best off tracing through it on your own to see how it works. To help you study, though, here are a few highlights worth mentioning: Inheritance versus delegation: The initial privacy example used inheritance to mix in a set of traits to catch accesses; inheritance makes this difficult because distinguishing accesses inside from outside the class is not straightforward. The version here uses delegation (embedding) instead, which is better suited to this task. Attributes accessed from outside the class are intercepted by the wrapper's overloading methods and delegated back if allowed; accesses from inside the class's methods are not intercepted and run normally. Decorator arguments: The class decorator accepts any number to name the private attributes; this really means the arguments are passed to `Private`, which returns the decorator, which remembers privates in an outer scope. State retention and enclosing scopes: There are actually there are three levels of state retention in play here: - `Private`'s arguments are retained in an outer scope for use in both `onDecorator` and `onInstance`. - `onDecorator`'s class argument is retained in an outer scope for use at instance-creation time. - The wrapped instance is retained as an instance attribute in

the onInstance proxy. Using dict and slots: setattr relies on dict to set the proxy's own wrapped attribute (it cannot assign directly without looping). It uses setattr to set attributes in the wrapped object, and getattr to fetch (since those may be inherited or in slots). As such, this works for virtual attributes, slots, properties, descriptors, and even classes with getattr. The wrapped class may use slots or properties; the proxy always has a dict.

中文翻译

(标题节对应书中的"实现细节"几个小节，下面逐节展开。)

继承与委托：最初的隐私示例在第 30 章，用继承混入一个 setattr 来捕获访问。继承让这件事变得困难，因为区分"从类内部访问"与"从类外部访问"并不直接。于是这里改用委托 (embedding) 替代继承：外部访问由代理的运算符重载方法拦截并委托回类本身 (验证通过时)；类内部的方法体 (通过 self) 访问正常执行——因为隐私在此版本中不被继承。

装饰器参数：类装饰器接受任意数量的参数以指定私有属性名。同样地，这意味着参数传给 Private，Private 返回装饰器，装饰器把 privates 表格以外层作用域引用"记住"。

状态保存与外层作用域：这段代码里实际有三级状态保存：

- Private 的参数 (在装饰发生前用到) → 保留为外层作用域引用，供 onDecorator 与 onInstance 一同使用。
- onDecorator 的类参数 (在装饰时用到) → 保留为外层作用域引用，供"创建实例时刻"使用。
- 被包装的实例 → 作为 onInstance 代理对象的实例属性保留。

dict 与 slots 及虚拟属性：setattr 依赖实例属性命名空间字典 dict，

为 `onInstance` 自身设置 `wrapped` 属性（直接赋值会陷入循环，必须写 `self.dict[attr]`）。对被包装对象则用 `setattr` 设置、用 `getattr` 抓取（因为属性可能存于对象本体或由它继承）。因此它对大多数类都适用——包括基于 `slots`、`property`、描述符甚至 `getattr` 的虚拟类级别属性。代理类本身始终有 `dict`（不用 `slots`），故“自己只管字典、被包装者用存储无关工具”的策略让代理类免于各种底层存储工具的限制。

深度理解

- 核心概念：“为什么用委托不用继承”的核心答案——区分内外部访问在委托下天然成立、在继承下几乎做不到。
- 底层实现：`setattr` 里“自设属性绕过 `setattr`”使用 `self.dict[attr]=...` 直写，规避死循环；代理自身的 `wrapped` 与“外来属性”走向两套逻辑。
- 设计原因：Python 哲学是“不加护栏”，作者刻意把私有放在“可选工具”的位置，而非语言层面的强约束。
- 实际问题：对 `slots` 类的委托依然有效——因为代理自己对 `dict` 的依赖在“代理层”而非被包装层。
- 初学者误区：在 `setattr` 里直接 `self.attr = value` 会引发无限递归——必须明白“赋值操作步步都经过 `setattr`”这个 Python 事实；这是本次实现最反直觉的一行。

代码分析

（关键岔路演示——为什么 `setattr` 要走 `dict`）

```
def __setattr__(self, attr, value):
    if attr == 'wrapped':
        self.__dict__[attr] = value    # __setattr__
    elif attr in privates:
        raise TypeError('private attribute change, ' + attr)
    else:
        setattr(self.wrapped, attr, value)    #
```

解释器执行链: `X.label = 'Hack' → onInstance.setattr('label', 'Hack') → 不在 privates → setattr(self.wrapped, 'label', 'Hack') → 委托到内嵌对象`。而 `self.dict['wrapped'] = ...` 直接写实例字典, 不再触发 `setattr`, 杜绝死循环。设计思想: 区分“管理代理自身”与“操作被包装对象”两条赋值通道, 是代理型工具的通用骨架。

39.31 Generalizing for Public Declarations (泛化出 Public 声明)

英文原文

Now that we have a Private attribute implementation, it's straightforward to generalize the code to allow for Public declarations too—they are essentially the inverse of Private declarations, so we need only negate the inner test. The example listed in Example 39-18 allows a class to use decorators to define a set of either Private or Public instance attributes—attributes of any kind stored on an instance or inherited from its classes—with the following semantics: Private declares attributes that cannot be fetched or assigned except for

om within the class's methods. Any name declared Private is inaccessible outside the class; any name not declared Private can be freely fetched or assigned from outside. Public declares attributes that can be fetched or assigned both from outside the class and within its methods. Any name declared Public is freely accessible; any name not declared Public cannot be accessed from outside. Private and Public declarations are mutually exclusive. When using Private, all undeclared names are Public; when using Public, all undeclared names are Private. They are opposites; new names can be assigned (and thus created) outside the class under Private, but not under Public. Again, study the code on your own. Note that this scheme adds a fourth level of state retention at the top: the validation function (faillf) is saved in an extra enclosing scope.

中文翻译

既然我们有了 Private 属性实现，泛化出 Public 声明就顺理成章——Public 本质上是 Private 的反面，只需取反内层测试即可。示例 39-18 允许一个类用装饰器定义一组"Private 或 Public 的实例属性"——即存储在实例上或从类继承的任何属性——其语义如下：Private（私有）声明"除了在类的方法代码之内，不能被抓取或赋值"的属性。任何声明为 Private 的名字都不能从类外访问；任何未声明为 Private 的名字都可以从类外自由抓取或赋值。Public（公共）声明"既可从类外也可从类内方法中抓取或赋值"的属性。任何声明为 Public 的名字都可自由访问；任何未声明为 Publ

ic 的名字都不能从类外访问。Private 与 Public 声明互斥。用 Private 时，所有未声明名字被视为 Public；用 Public 时，所有未声明名字被视为 Private。它们本质上相反——在 Private 下，类外可以新赋值并创造新名字；在 Public 下则不行。再次提醒：请自行研读代码。注意本方案在最顶层又添加了第四级状态保存：验证函数 (faillf) 保存在额外的外层作用域里（供 lambda 使用）。

深度理解

- 核心概念：把"名单判定"抽象成策略函数 failIf——Private 传" $\in$ 名单"，Public 传"名单"，一正一反两个入口共用一套骨架。

- 底层实现：accessControl(failIf) 工厂把判定函数保存在最外层作用域；Private(attrs) 与 Public(attrs) 只是两个薄壳。

- 设计原因：第四级作用域让"校验策略"与"代理逻辑"解耦——将来换规则（如正则匹配）只需换 failIf。

- 实际问题：把"默认拒绝 vs 默认放行"两种安全模型用同一实现切换，白名单/黑名单通用。

- 初学者误区：忘了"新名字"规则——Private 模式下 $X.newattr = 1$ 是合法的（未声明即公开），Public 模式下同样操作被拒。这种不对称最易踩坑。

代码分析

```

# 39-18access2.py
def accessControl(failIf):
    def onDecorator(aClass):
        class onInstance:
            def __init__(self, *args, **kargs):
                self.__wrapped = aClass(*args, **kargs) # __X ...
            def __getattr__(self, attr):
                trace('get:', attr)
                if failIf(attr): #
                    raise TypeError('private attribute fetch, ' + ...
                else:
                    return getattr(self.__wrapped, attr)
            def __setattr__(self, attr, value):
                trace('set:', attr, value)
                if attr == '_onInstance__wrapped': #
                    self.__dict__[attr] = value
                elif failIf(attr):
                    raise TypeError('private attribute change, ' + ...
                else:
                    setattr(self.__wrapped, attr, value)
        return onInstance
    return onDecorator

def Private(*attributes):
    return accessControl(failIf=(lambda attr: attr in attributes))...
def Public(*attributes):
    return accessControl(failIf=(lambda attr: attr not in attribut...

```

REPL 演示: @Private('age') 的 Person, X.name 可读写、X.age 读写均被拒; @Public('name') 时则 X.name 可读写、X.age 被拒。设计思想: 判定逻辑提升为"一等公民" (函数), 语义仅由一个 lambda 的取反决定——这是泛化 (generalization) 的教科书式范例。

39.32 Implementation Details II (实现细节 (二))

英文原文

To help you analyze Example 39-18's code, here are a few final notes on this version. Using "X" pseudoprivate names: This version also makes use of Python's X pseudoprivate name mangling feature to localize the wrapped attribute to the proxy class by automatically prefixing it with the class's name. This avoids collisions with a wrapped attribute that may be used by the real, wrapped class. It's not quite "privacy," though, because the mangled version can be used freely outside the class. Notice we must use the expanded name string, 'onInstancewrapped', as an admin-name test value in setattr. Breaking privacy: Though it implements access controls, it is possible to subvert them trivially—e.g., by fetching through the fully mangled name `bob.onInstancewrapped.pay`. If you have to try that hard to break them, these tools probably suffice for intended roles. Of course, privacy can generally be subverted in other languages too (e.g., `#define private public` in C++). Python is about enabling, not controlling; privacy is a tool best used sparingly (if at all). Decorator trade-offs: The chief downsides of this wrapper-based approach are that attribute access incurs an extra call, and instances of decorated classes are not

really instances of the original class—`X.class` or `isinstance(X, C)` reveal they are instances of the wrapper class. Unless you plan to do introspection, the type issue is irrelevant; and the extra call may apply mostly to development time, as it's possible to remove decorations automatically via `-O`.

中文翻译

为了帮你分析示例 39-18 的代码，这里给出该版本的几条最后备注。使用 X 伪私有名称：这个版本还利用了 Python 的 X 伪私有名（name mangling，名字改编）特性——给 wrapped 属性自动加上类名前缀，把它局部化到代理控制类里。这避免了与真实被包装类可能使用的 wrapped 属性冲突。不过这并非真正的“隐私”——变形后的全名在类外仍可自由使用。注意：在 `setattr` 里，我们必须用展开后的全名串 `'onInstancewrapped'` 作为管理名测试值，因为那才是 Python 改写后的名字。破坏隐私：尽管本示例确实实现了属性访问控制，但可以轻易地绕过——例如通过展开后的全名显式抓取（`bob.pay` 可能不行，但 `bob.onInstancewrapped.pay` 可以）。如果必须费这么大力气才能绕过，那这些工具大概已足以胜任预期角色。当然，其他语言里隐私通常也可以被绕过（例如某些 C++ 实现里 `#define private public` 可能有效）。只要源代码可被修改，滴水不漏的访问控制永远是白日梦。更根本的是，Python 讲的是“赋能（enabling）”，不是“控制（controlling）”；隐私是一种最好少用（甚至不用）的工具。装饰器的权衡（trade-offs）：这个以及任何基于包装器的方法，其主要缺点是属性访问多一次额外调用，而且被装饰类的实例并不是原装饰类

的真正实例——用 `X.class` 或 `isinstance(X, C)` 测试，你会发现它们是包装器类的实例。除非你打算对对象类型做内省（introspection），类型问题无关紧要；额外调用也主要发生在开发期——如后所述，用 `-O` 可以自动移除装饰。

深度理解

- 核心概念：伪私有名（wrapped → onInstancewrapped）解决"名字冲突"；但这不是安全机制——Python 的隐私向来是"君子协定"。
- 底层实现：类体内的 `attr` 会被编译器改写为类名`attr`，类外无此改写；因此管理名测试必须用变形后的全串。
- 设计原因：作者的态度——"隐私工具"用于开发期防手滑，不是安全边界；C++/Java 的 `private` 同样可被暴力绕过。
- 实际问题：权衡清单——每次属性访问多一次调用（性能）、`isinstance` 失效（类型内省）、`-O` 可整体移除（优化通道）。
- 初学者误区：把"装饰后的代理实例"当作原类实例用 `isinstance` 判断而得到 `False`，误以为程序有 bug；这是代理模式的固有性质而非缺陷。

代码分析


```
#
class onInstance:
    def __init__(self, *args, **kargs):
        self.__wrapped = aClass(*args, **kargs)    #
        # self._onInstance__wrapped = aClass(*args, **kargs)

# ""
bob._onInstance__wrapped.pay    #
#
# python -O main.py __debug__ False
```

设计思想：把"防止意外"与"防止恶意"区分开——Python只承诺前者。理解这一点，你就同时理解了"为什么有这种工具"与"为什么它不能当防火墙"。

39.33 Delegating Built-In Operations (委托内置运算)

英文原文

As is, this section's examples work as planned for methods and other attributes fetched explicitly by name. As with most software, though, there is always room for improvement. Most notably, this tool turns in mixed performance on operator-overloading methods if they are used by client classes. Specifically, the proxy class fails to validate or delegate operator-overloading methods fetched implicitly by built-in operations unless such methods are redefined in the proxy. Clients that do not use operator overloading are fully supported, but others may require additional code. It's un

clear that operator-overloading methods should be validated as private or public, but they are a part of an object's interface and should at least be routed to wrapped objects that define them. The basic issue is easy to demo. A class that overloads print calls and + expressions works normally. But objects that implement built-in operations fail in our proxy classes because built-in operations skip instance-level lookup protocols like getattr, and instead search namespaces of classes. In this session, the first two explicit fetches of sum and add are kicked out as privates as they should be. Because the last two implicit fetches of print and + aren't caught by the proxy, they are never delegated to the wrapped Tally object. The print works at all only because it runs an object default to print the proxy itself.

中文翻译

就现状而言，本节示例对"显式按名字抓取的方法和其他属性"能按计划工作。但和大多数软件一样，总有改进空间。最值得注意的是：如果客户端类使用了运算符重载方法，这个工具的表现就参差不齐。具体来说，除非运算符重载方法在代理中被重新定义，否则代理类无法校验或委托内置运算隐式抓取的那些运算符重载方法。不使用运算符重载的客户端完全受支持，但其他客户端可能需要额外代码。运算符重载方法是否应被当作私有或公共来校验并不清楚，但它们是对象接口的一部分，至少应被路由到定义了它们的被包装对象。基本问题很容易演示。重载了 print 调用与 + 表达式

的类本来工作正常。但实现内置运算的对象在我们的代理类里会失败——因为内置运算会跳过 `getattr` 这类实例级查找协议，转而去类的命名空间里搜索。在本会话中，前两次对 `sum` 和 `add` 的显式抓取如预期那样被当作私有点名拒绝。而后两次 `print` 与 `+` 的隐式抓取没被代理接住，因此永远不会被委托给被包装的 `Tally` 对象。`print` 之所以还能工作，只是因为它运行的是 `object` 的默认行为——打印代理对象自身。

深度理解

·核心概念：内置运算的查找协议绕过实例层——`X + 5` 直接在 `type(X)` 的类命名空间找 `add`，根本不会落入 `getattr`。这是 Python 属性协议的一个著名怪癖。

·底层实现：`getattr` 只在“实例正常属性查找链全失败”时触发；而内置运算走“类型（类）层面”的特殊槽位查找，两套通道并行不悖。

·设计原因：性能考量与历史包袱使然——Python 为内置运算保留快速路径。理解它，才能解释“代理为何接不住 `+`”。

·实际问题：任何“委托代理”（ORM、RPC、mock）都要为 `str`、`add` 等显式补一层，否则内置语法失效。

·初学者误区：`print(X)` 能输出“对象地址”，让人误以为委托成功；其实那是 `object` 的默认 `str`，被包装对象的 `str` 从未被调用。

代码分析

```
>>> class Tally:
...     def __init__(self): self.sum = 0
...     def __str__(self): return f'Tally: {self.sum}'
...     def __add__(self, add): self.sum += add
>>> X = Tally()
>>> print(X)          # Tally: 0
>>> X + 5              # X.__add__(5)
>>> print(X)          # Tally: 5

>>> from access2 import Private
>>> @Private('sum', '__add__')
... class Tally: ...
>>> X = Tally()
>>> X.sum              # → private attribute fetch, sum
>>> print(X)          # →
>>> X + 5              # → unsupported operand type(s)
```

执行链对比：X.sum 走实例查找链 → 落入 getattr → 校验；print(X) 走类层面 str 槽位 → 代理类没定义 → 退回 object.str；X + 5 同理在代理类上找不到 add 而抛类型错误。设计思想：两条查找通道（实例协议 vs 类槽位）是理解 Python 全部"魔法方法"行为的地基。

Workaround 1: Coding operator-overloading methods inline（方案一：内联重定义）

英文原文

The most straightforward way to support built-ins in delegation proxies is to redefine operator-overloading names that may appear in embedded objects. This

creates some code redundancy, but it isn't impossibly onerous; it can be automated with tools or superclasses; and it can choose to run or skip validations for operator-overloading names declared Private or Public, depending on redefinitions' routing. For instance, the partial listing of Example 39-19 sketches an inline redefinition approach—it catches and delegates built-ins by adding method definitions to the proxy itself for every operator-overloading method a wrapped object may define. It adds just four operation interceptors to illustrate, but others are similar.

中文翻译

在委托代理中支持内置运算最直接的方法，是把嵌入对象可能出现的运算符重载名字在代理里重定义一遍。这会造成一些代码冗余，但并非无法承受；可以用工具或超类自动化；也可以根据重定义的路由选择“对声明为 Private/Public 的运算符重载名做校验或跳过校验”。例如，示例 39-19 的局部清单勾勒了“内联重定义”方案——给代理本身添加方法定义，逐个捕获并委托被包装对象可能定义的运算符重载方法。这里只加了四个运算拦截器示意，其他同理。

深度理解

·核心概念：既然内置运算在代理类上找不到名字，就“手动在代理类里摆好这些名字”，让它们显式转发到 `self.wrapped`。

·底层实现：`add` 等在代理类命名空间里存在后，内置运算的类级查找命中，直接调用代理方法，代理方法内再调被

包装对象。

·设计原因：直接、可预期、零元编程；代价是"每运算符一行样板"。

·实际问题：绕过校验直接转发（跳过 getattr 验证），适合"信任被包装对象"的场合。

·初学者误区：以为加一个 add 就能接住所有运算——rad d、iadd、比较符、len.....清单很长，通用代理需要穷举

。

代码分析

```
# 39-19access_builtins_inline_direct.py
def accessControl(failIf):
    def onDecorator(aClass):
        class onInstance:
            def __init__(self, *args, **kargs):
                self.__wrapped = aClass(*args, **kargs)
            #
            def __add__(self, other):
                return self.__wrapped + other          # getattr()/_...
            def __str__(self):
                return str(self.__wrapped)              # self.__wra...
            def __getitem__(self, index):
                return self.__wrapped[index]
            def __call__(self, *args, **kargs):
                return self.__wrapped(*args, **kargs)
            # .....
            #
            def __getattr__(self, attr): ...
            def __setattr__(self, attr, value): ...
        return onInstance
    return onDecorator
```

执行链: `print(X) → 类级找 str → 命中代理的 str → 返回 str(self.wrapped) → 触发被包装对象的 str`。设计思想: 把"内置运算"显式桥接进代理——本质是"按需列举接口"。

Workaround 2: In superclasses (方案二: 写在超类/混入类中)

英文原文

More usefully, the prior section's added methods can be added by a common superclass. Given that there are dozens of such methods, an external class may be better suited to the task, especially if it is general enough to be used in any such interface-proxy class. To demo, the superclass of Example 39-20 catches built-ins and reroutes to the wrapped object directly again. It's largely just a repackaging of the prior section's inline scheme, but as a separate class it requires a proxy attribute named `wrapped`, giving access to the embedded object. The decorator itself must use this name instead of `wrapped` in self references, and sans mangling in `setattr`. Alternatively, the superclass in Example 39-21 catches built-ins and reroutes them down through the subclass `getattr` to apply its access controls to the operation's method name. It requires that operator-overloading names be non-Private (or Public) per the decorator's arguments if they are to be run

, but it treats the implicit fetches of built-in operations the same as explicit-name fetches, and no wrapping is required in subclasses.

中文翻译

更有用的是，把上一节的“新增方法”放进一个公共超类里。既然这类方法有几十个，外部类可能更适合这个任务——尤其是当它通用到“任何此类接口代理类都能用”时。演示一下：示例 39-20 的超类再次直接捕获内置运算并转发给被包装对象。它大体只是把上一节的内联方案重新打包，但作为独立类，它要求代理有一个名为 `wrapped` 的属性以访问嵌入对象；装饰器自身在 `self` 引用里也必须用它（不用 `wrapped`），`setattr` 里也不涉及名字变形。另外，示例 39-21 的超类捕获内置运算后，会经由子类的 `getattr` 再路由下去，从而把访问控制施加到该运算的方法名上。它要求运算符重载名字按装饰器参数是“非 Private（即 Public）”才能运行；它把内置运算的隐式抓取与显式名字抓取同等对待，而且子类无需 `wrapped`。

深度理解

- 核心概念：把几十个 X 样板提升为混入超类（mixin）——`class onInstance(BuiltinsMixin)`，一处编写、处处继承。
- 底层实现：示例 39-20 直连 `self.wrapped`（跳过校验）；示例 39-21 改为 `self.getattr('add')(other)`（走校验），只是转发路径不同。
- 设计原因：MRO 查找让子类自动获得全套内置桥接，既

省样板又保证一致性。

·实际问题：需要一个共享的 wrapped 名字约定，且不能与包装类自己的属性冲突——通用工具设计时命名即契约

。

·初学者误区：只记得"加方法"，却忘了"走哪条通道"决定是否经过校验——两条 mixin 路线代表两种语义，用途不同。

代码分析

```
# 39-20access_builtins_mixin_direct.py
class BuiltinsMixin:
    def __add__(self, other):
        return self._wrapped + other      # _wrapped __geta...
    def __str__(self):
        return str(self._wrapped)
    def __getitem__(self, index):
        return self._wrapped[index]
    def __call__(self, *args, **kargs):
        return self._wrapped(*args, **kargs)
# class onInstance(BuiltinsMixin): _wrapped

# 39-21access_builtins_mixin_getattr.py
class BuiltinsMixin:
    def __add__(self, other):
        return self.__getattr__('__add__')(other)  #
    def __str__(self):
        return self.__getattr__('__str__')()      #
    def __getitem__(self, index):
        return self.__getattr__('__getitem__')(index)
    def __call__(self, *args, **kargs):
        return self.__getattr__('__call__')(*args, **kargs)
```

执行链 (39-21 版)：X + 10 → 类级找到 mixin 的 add

→ `self.getattr('add')` → 若 `add` 被声明 `Private` 则抛异常；否则委托 `wrapped.add(10)`。设计思想：让内置运算"绕回" `getattr`，一举统一"显式/隐式"两条抓取通道的校验语义。

Workaround 3: Generating operator-overloading descriptors (方案三：生成运算符重载描述符)

英文原文

Finally, all of the inline and mix-in workarounds for built-ins we've seen so far code each operator-overloading method explicitly, and intercept the actual call issued for the operation, including its arguments. That makes them responsible for completing the operation, whether by operation syntax or equivalent calls. With an alternative coding, we could instead intercept only the attribute fetch preceding the call by using the class-level descriptors of the prior chapter. Moreover, because all such descriptors will run the same, they can be generated automatically from a list of method names. Example 39-22 shows one way to code this scheme. Like Example 39-21, it routes built-in operations through the decorator's validations logic. Recall that the `exec` built-in by default runs a string of code as if it was pasted where `exec` appears. Hence, the loop at the end of this mix-in class is equivalent to the following statements, run in the mix-in class's local scope

e: add = ProxyDesc("add"); str = ProxyDesc("str"); and so on. The net effect creates inherited descriptor instances that respond to initial name lookups by fetching from the wrapped object in get rather than catching the later operation call itself.

中文翻译

最后，目前看到的所有内联与混入内置运算修补方案，都是显式编写每个运算符重载方法，并拦截"为运算发出的实际调用（含参数）"。这让它们负有"完成运算"之责——无论用运算语法还是等价调用。另一种编码可以只用前一章的类级描述符，只拦截"调用之前的那次属性抓取"。而且，因为所有这些描述符行为一致，它们可以从一个方法名列表自动生成。示例 39-22 演示了该方案的一种写法。与示例 39-21 一样，它让内置运算经过装饰器的校验逻辑。回忆一下：exec 内建函数默认把一串字符串代码当作"粘贴在 exec 出现处"来执行。因此，该混入类末尾的循环等价于下列在混入类局部作用域中执行的语句：add = ProxyDesc("add")、str = ProxyDesc("str").....依此类推。净效果是创建了一组被继承的描述符实例：它们响应"首次名字查找"，在 get 里从被包装对象取货，而不是去拦截之后的运算调用本身。

深度理解

·核心概念：把"拦截调用"换成"拦截属性抓取"——用描述符在类级把 add 等名字占位，抓取时经 get 注入校验与委托。

·底层实现：exec(f'{attr} = ProxyDesc("{attr}")') 在类体里批量生成类属性；每个 ProxyDesc.get 调用 instance.

getattr(self.attrname)。

·设计原因：从方法名列表生成描述符，几十个运算符零样板——把"元编程"用到极致，但作者直白警告这是"最隐晦、最复杂"的方案。

·实际问题：运维通用代理时，"列表驱动"比"手工抄写"更不易漏；代价是可读性。

·初学者误区：被 exec 吓到或滥用——exec 是运行时执行字符串代码，等同"在类体里粘贴语句"，本文用它正是生成重复样板的正规场合。

代码分析

```
# 39-22access_builtins_mixin_desc.py
class BuiltinsMixin:
    class ProxyDesc:
        #
        def __init__(self, attrname):
            self.attrname = attrname
        def __get__(self, instance, owner):
            return instance.__getattr__(self.attrname) #

    builtins = ['add', 'str', 'getitem', 'call'] #
    for attr in builtins:
        exec(f'__{attr}__ = ProxyDesc("__{attr}__")') #
# class onInstance(BuiltinsMixin): ...

#
# __add__ = ProxyDesc("__add__")
# __str__ = ProxyDesc("__str__")
# ...

#
# def __get__(self, instance, owner):
#     return getattr(instance._wrapped, self.attrname) # ...
```

执行链: $X + 10 \rightarrow$ 类级找到继承来的 `add` 描述符 $\rightarrow$ 调用它触发 `ProxyDesc.get(X, onInstance) \rightarrow X.getattr('add')` $\rightarrow$ 校验 (`Private` 则抛错; 否则委托被包装对象) $\rightarrow$ 得到的可调用再被 `+` 运算调用。设计思想: 描述符把"抓取时刻"与"调用时刻"分离, 前者挂接校验、后者自然完成——比"拦截调用"更早一步介入, 也更容易批量化。

39.34 Example: Validating Function Arguments (案例二: 验证函数参数)

英文原文

As a final example of the utility of decorators, this section develops a function decorator that automatically tests whether arguments passed to a function or method are within a valid numeric range. It's designed to be used during either development or production, and it can be used as a template for similar tasks (e.g., argument type testing, if you must). Again, this example is largely self-study content with a limited narrative; read the code for more details.

中文翻译

作为装饰器效用的最后一个示例, 本节开发一个函数装饰器: 自动测试传给某函数或方法的参数是否处于有效数值区间内。它设计为可在开发期或生产期使用, 也可作为同类任务(比如不得不做的参数类型测试)的模板。同样, 本示例大

体属于自研读内容、叙述有限——更多细节去看代码。

深度理解

- 核心概念：参数范围校验器的目标与前一节的"属性隐私"恰恰互补——前者管"传进函数的值"，后者管"访问属性的值"。
- 底层实现：用函数内省 (`func.code.covarnames`) 获取"期望的参数名列表"，作为参数名与位置的映射依据。
- 设计原因：把校验从"每个调用点"抽到"一处声明"，同时利用 `debug/-O` 一键关闭，做到"开发期校验、生产期零开销"。
- 实际问题：金融/数据代码里的百分比、日期字段（如月 1-12、日 1-31、年合理范围）边界校验的通用模板。
- 初学者误区：以为"校验参数"必须手写 `if`；本书展示了 `@rangetest(percent=(0.0,1.0))` 这样一行声明式的优雅替代。

39.35 The Goal (目标)

英文原文

In the object-oriented tutorial of Chapter 28, we wrote a class that gave a pay raise to objects representing fictitious people, based upon a passed-in percentage.

```
class Person:
    def giveRaise(self, percent):
        self.pay = int(self.pay * (1 + percent))
```

There, we noted that if we wanted robust code, it would be a good idea to check t

he percentage to make sure it's not too large or too small. We could implement such a check inline with if or assert statements in the method itself. However, this approach cluttered the method with inline tests that will probably be useful only during development. For more complex cases, this can become tedious. Perhaps worse, if the validation logic ever needs to change, there may be arbitrarily many inline copies to find and update. A more useful and interesting alternative would be to develop a general tool into perform range tests automatically. A decorator approach makes this explicit and convenient, and easy to disable once development is complete:

```
class Person:
    @rangetest(percent=(0.0, 1.0))
    # Use decorator to validate
    def giveRaise(self, percent):
        self.pay = int(self.pay * (1 + percent))
    Notice that the goal here is different from the attribute validations coded in the prior chapter's final example: we validate the values of function arguments when passed rather than attribute values when accessed.
class Person:
    def giveRaise(self, percent):
        self.pay = int(self.pay * (1 + percent))
class Person:
    @rangetest(percent=(0.0, 1.0))
    # 用装饰器校验
    def giveRaise(self, percent):
        self.pay = int(self.pay * (1 + percent))
```

中文翻译

在第 28 章的面向对象教程里，我们写过一个方法：根据传入的百分比给代表虚构人物的对象加薪。那里我们提到：如果想让代码健壮，最好检查这个百分比，确保它不会过大或

过小。我们可以在方法里用内联的 if 或 assert 实现这种检查。但这种做法用只在开发期可能有用的内联测试把方法塞满，对于更复杂的情形会变得很累人。更糟的是，如果校验逻辑将来需要修改，可能有任意多处内联副本要去找、去改。更有用也更妙的替代是开发一个通用工具，自动为我们做范围测试。装饰器方案让这变得显式而便捷，一旦开发完成也很容易禁用：注意：这里的目标与上一章最终示例里的属性校验不同——我们是校验"传入的参数值"，而不是"访问时的属性值"。

深度理解

- 核心概念：两条校验路线的对照——上一章校验"属性值被访问时"，本章校验"参数值被传入时"；都可用装饰器统一收口。
- 底层实现：与方法内联 `if percent < 0 or percent > 1` 等价，装饰器版把这些逻辑移出方法体、集中为声明。
- 设计原因：解决"内联校验三宗罪"——污染业务方法、开发期专用、变更时几十处同步更新。
- 实际对比：内联版改变量时会有"处处找、处处改"的维护噩梦；装饰器版只改一处。
- 初学者误区：不清楚"校验何时发生"——装饰器校验发生在每次调用进入原函数之前（靠包装器），而非函数体内。

39.36 A Basic Range-Testing Decorator for Positional Arguments (基础版：仅支持位置参数)

英文原文

To keep things simple, we'll begin by coding a decorator that works only for positional arguments and assumes they always appear at the same positions in every call; they cannot be passed by keyword name because this invalidates the positions declared in the decorator. Example 39-23 is our first-cut checker. This code is mostly a rehash of the coding patterns we explored earlier: decorator arguments, nested scopes for state retention, and nested `def` to support both functions and methods. When used on a method, `onCall` receives the subject class's instance in the first item in `args` and passes this along to `self`; explicitly passed argument numbers in the decorator line start at 1 in that case to accommodate the implicit `self`. New here is the `debug` built-in variable: Python sets it to `True` unless the program runs with the `-O` optimize flag. When `debug` is `False`, the decorator returns the original function instead of a wrapper to avoid extra calls and their performance penalty—automatically removing a augmentation logic without requiring you to physically remove the decoration lines.

中文翻译

为了保持简单，我们先写一个只适用于位置参数的装饰器，并假设它们在每次调用中总出现在相同位置；不能用关键字名字传它们，因为那会让声明的位置号失效。示例 39-23 是第一版检查器。这段代码基本是前面讲的编码模式的旧瓶新酒：装饰器参数、嵌套作用域做状态保存、嵌套 `def` 以同时支持函数与方法。用于类的某个方法时，`onCall` 会在 `args` 的第一项收到主题类实例并转发给 `self`；此时装饰器行里显式写的参数编号要从 1 开始（0 位留给了隐式 `self`）。本节新引入了 `debug` 内建变量：除非程序以 `-O` 优化标志运行，Python 把它设为 `True`。当 `debug` 为 `False` 时，装饰器返回原函数而非包装器，以避免额外调用及其性能代价——自动移除增强逻辑，无需你从源码里手工删行。

深度理解

- 核心概念：位置参数校验的第一版——`@rangetest((1, 0, 120))` 意为“第 1 个（从 0 数）位置参数必须在 0..120”。
- 底层实现：三层结构 `rangetest(argchecks) → onDecorator(func) → onCall(args)`；`onCall` 逐个处理 `(ix, low, high)` 检查 `args[ix]`。
- 关键设计：`debug/-O` 开关——`python -O main.py` 时校验自动失效、性能零损耗，这是“开发校验、生产直通”的力作。
- 实际问题：方法场景：`@rangetest([1, 0.0, 1.0])` 因为 `args[0]` 是 `self`，参数号从 1 起。

·初学者误区：调试期忘了"位置参数版不能接受关键字传参"——一旦 `persinfo(name='Bob', age=45)` 传入，位置语义立刻错乱。

代码分析

```
# 39-23rangetest0.py
def rangetest(*argchecks):
    def onDecorator(func):
        if not __debug__:
            return func
        else:
            def onCall(*args):
                for (ix, low, high) in argchecks:
                    if args[ix] < low or args[ix] > high:
                        errmsg = f'Argument {ix} not in {low}..{hi...'
                        raise TypeError(errmsg)
                return func(*args)
            return onCall
        return onDecorator

# 39-24rangetest0_test.py
@rangetest((1, 0, 120))
def persinfo(name, age):
    print(f'{name} is {age} years old')

class Person:
    @rangetest([1, 0.0, 1.0])
    def giveRaise(self, percent):
        self.pay = int(self.pay * (1 + percent))

persinfo('Bob Smith', 45)
#persinfo('Bob Smith', 200)
sue.giveRaise(.10)
#sue.giveRaise(1.10)
```

执行链：@rangetest(...) 时，若 debug 为 True（正常不加 -O），把函数换成 onCall；调用时 onCall 遍历 argchecks，按位置索引校验，越界即 raise TypeError。运行 python -O 时 debug 为 False，直接返回 func——校验逻辑被“自动摘除”。设计思想：校验开关挂在语言级优化标志上，零成本生产。

39.37 Generalizing for Keywords and Defaults（泛化：支持关键字与默认值）

英文原文

The prior version illustrates the basics we need to employ, but it's fairly limited—it supports validating arguments passed by position only and it doesn't validate keyword arguments; it also does nothing about arguments with defaults that may be omitted. That's fine if all your arguments are passed by position and never defaulted, but less than ideal in a general tool. The level up in Example 39-25 does better. By matching the wrapped function's expected arguments against the actual arguments passed in a call, it supports range validations for arguments passed by position or keyword, and it skips testing for default arguments omitted in the call. Arguments to be validated are specified by keyword arguments to the decorator itself. The final version handles positional, keyword, and omitted d

efaults. Let's look at the test code, which shows how the decorator is used, then at its output.

中文翻译

前面的版本拿出了我们所需的基础，但相当受限——只支持按位置传入的参数校验，不校验关键字参数（事实上它假设没有以关键字方式传入的参数）。它对“有默认值、可能被省略”的参数也毫无作为。如果所有参数都按位置传递、从不省略，这没什么问题；可作为通用工具就不太理想了。示例 39-25 的升级版做得好些。它把被包装函数期望的参数与一次调用实际传入的参数对照起来，从而支持按位置或按关键字传入两类参数的区间校验，并跳过“调用中被省略、取默认值”的参数。要校验的参数由装饰器自身的关键字参数指定；实际调用时既可按位置也可按关键字传值，带默认值的参数可以省略。最终版同时支持位置参数、关键字参数与省略默认值。下面是测试代码（展示装饰器用法）及其输出。

深度理解

- 核心概念：从面向位置到面向名字——用函数内省获知“期望参数名列表”，从而把“装饰器声明的参数名”映射到“实际调用传入的值/位置”。
- 底层实现：`func.code.covarnames[:coargcount]` 截取“前 N 个局部变量”作为期望参数名；`pargs` 的前 N 个即位置实参。
- 设计原因：固定位置方案太脆；按名字匹配方案柔性覆盖“位置/关键字/默认省略”三种传法。

·解决问题：一版通用工具通吃三种传参风格，甚至 `birthday(8, D=31, Y=2024)` 这种混合调用。

·初学者误区：以为"默认值参数也必须校验"。设计正确行为是省略即跳过——`omitargs(1, 2, 3)` 时 `d` 没传，就不检查 `d`。

代码分析

```
# 39-25rangetest.py
trace = True

def rangetest(**argchecks):
    #
    def onDecorator(func):
        # onCall  func  argch...
        if not __debug__:
            return func
        #
        else:
            funcname = func.__name__
            funccode = func.__code__
            funcargs = funccode.co_varnames[:funccode.co_argcount]...
            def onCall(*pargs, **kargs):
                # pargs  N
                # kargs
                positionals = funcargs[:len(pargs)]
                errmsg = lambda *args: '%s argument "%s" not in ...
                for (argname, (low, high)) in argchecks.items():
                    if argname in kargs:
                        #
                        if kargs[argname] < low or kargs[argname] ...
                            raise TypeError(errormsg(funcname, arg...
                    elif argname in positionals:
                        #
                        position = positionals.index(argname)
                        if pargs[position] < low or pargs[position]...
                            raise TypeError(errormsg(funcname, arg...
                    else:
                        if trace:
                            #
                            print(f'-Argument "{argname}" defaulte...
                return func(*pargs, **kargs)
            # ...
```

```
return onCall
return onDecorator
```

(测试: @rangetest(M=(1,12), D=(1,31), Y=(0,2024))
装饰 birthday(M, D, Y)——birthday(8, D=31, Y=2024)
) 合法; persinfo(age=40, name='Pat') 按名传参照样通过; 省略默认值的调用打印 -Argument "d" defaulted
并跳过检查。设计思想: 函数自省 + 星号收集参数, 构成了"名字位置"双向映射的最简实现, 是 Python 自省元编程的经典示范。

39.38 Implementation Details (实现细节)

英文原文

This range-tester decorator's code relies on both introspection APIs and subtle constraints of argument passing. To be fully general, we could try to mimic Python's argument-matching logic in its entirety, but that's too much complexity for our tool and is prone to change over time. It would be better if we could match the names of testable arguments given to the decorator against the names of actual arguments expected by the function. Function introspection: The introspection API available on function objects and their associated code objects has exactly the tool we need. The set of expected argument names is simply the first N variable names attached to a function's code object: `func.code.coargnames[:func.code.coargcount]`. Arg

ument assumptions: Given the expected argument names, the solution relies upon two constraints on ordering: - At the call, all positional arguments appear before all keyword arguments. - In the def, all nondefault arguments appear before all default arguments. That is, a nonkeyword argument cannot follow a keyword argument at a call, and a nondefault argument cannot follow a default argument in a definition. All name=value syntax must appear after any simple name in both places. We can also assume the call is valid in general—that all arguments either receive values or are omitted to pick up defaults. This won't necessarily hold, but since the call would fail here anyway, we might as well let Python's own argument matching detect invalid calls. Matching algorithm: Given these constraints, for a call being intercepted we can deduce: 1. Let N be the number of passed positional arguments (the length of the pargs tuple). 2. All N positional arguments must match the first N expected arguments, by Python's call ordering rules. 3. To obtain the names of arguments passed by position, slice the list of all expected arguments up to N. 4. Any arguments after the first N expected arguments either were passed by keyword or were defaulted by omission. 5. For each argument name to validate: (a) if in kargs, it was passed by name; (b) if in the first N expected, it was passed by position; (c) otherwise, assume it was omitted and defaulted and need not be checked.

中文翻译

这个区间测试装饰器的代码依赖两样东西：自省 (introspection) API，以及参数传递顺序的微妙约束。要做到完全通用，可以尝试完整模拟 Python 的参数匹配逻辑，但对我们的工具来说这过于复杂，而且随时间易变。更好的办法是：把"给装饰器的可测试参数名"与"函数期望达到的实际参数名"对照起来。函数内省 (function introspection)：函数对象及其关联代码对象上的内省 API，正好提供我们需要的工具。期望参数名集合，就是函数代码对象上附着的前 N 个变量名连接：`func.code.coarnames[:func.code.coargcount]`。演示：`def func(a, b, c, e=True, f=None)` 加上局部变量 `x, y` 后，`conlocals=7`、`coarnames=('a','b','c','e','f','x','y')`，而前 `coargcount=5` 个是 `('a','b','c','e','f')`。参数假设：有了期望参数名集合，我们的解法依赖两条 Python 强加的排序约束 (第 18 章已述)：- 在调用处，所有位置参数都在所有关键字参数之前。- 在定义处，所有非默认参数都在所有默认参数之前。也就是说：调用处，非关键字参数不能跟在关键字参数之后；定义处，非默认参数不能跟在默认参数之后。任何 `name=value` 语法都必须出现在两处所有简单名字之后。我们还可以假设本次调用总体上是合法的——所有参数要么收到值，要么故意省略取默认值。这条假设不必然成立，但反正调用在这里也会失败，不如让 Python 自己的参数匹配逻辑替我们发现非法调用。匹配算法 (matching algorithm)：在这些约束下，对一次被拦截的调用可以按如下推演：1. 令 `N` 为传入的位置参数个数，即 `pargs` 元组的长度。2. 全部 `N` 个位置参数，按 Python 调用排序规则，必须与前 `N` 个期望参数匹配。3.

为得到"按位置传入的参数名"，把全部期望参数列表切片到第 N 个。4. 前 N 个期望参数之后的任何参数，要么按关键字传入，要么因省略而取了默认值。5. 对每个待校验参数名：(a) 若在 kargs 里 → 按名传入，作弊索引即得值；(b) 若在前 N 个里 → 按位置传入，期望列表里的相对位置即它在 pargs 里的相对位置；(c) 否则假设在调用中被省略、取了默认值，无需检查。

深度理解

·核心概念：参数匹配算法的根在两条排序不变量（位置先于关键字——call 侧；非默认先于默认——def 侧），本书用它们省掉了"模拟整个匹配引擎"的苦役。

·底层实现：covarnames 给出全部局部变量（含形参与函数体局部），coargcount 标出"前 N 个是形参"；配合调用处 pargs/kargs 的收集，恰好能完成"名字 位置 值"三方对账。

·设计原因：不模拟 CPython 的匹配（易碎、难维护），而是利用两条排序规则推理出必然——算法的优雅老师。

·解决实际：三种传参风格混合的任意调用（birthday(8, D=31, Y=2024)、omitargs(1, b=2, d=4)），都能正确找到待校验参数的值。

·初学者误区：covarnames 不只是参数——它还包含函数体内声明的局部变量（本例 x、y），必须靠 coargcount 截断，否则会误把局部变量当作形参。

代码分析

```
>>> def func(a, b, c, e=True, f=None): #
...     x = 1 #
...     y = 2
>>> code = func.__code__ #
>>> code.co_nlocals
7
>>> code.co_varnames #
('a', 'b', 'c', 'e', 'f', 'x', 'y')
>>> code.co_varnames[:code.co_argcount] # <= N
('a', 'b', 'c', 'e', 'f')

def catcher(*pargs, **kargs): print(f'{pargs}, {kargs}')
catcher(1, 2, c=3, d=4, e=5) # (1, 2), {'c': 3, 'd': 4...
```

执行解读：调用侧 `pargs` 收集"位置实参元组"，`kargs` 收集"关键字命名参数字典"；被包装侧的 `co_varnames[:code.co_argcount]` 提供"期望形参名元组"。装饰器在两者之间做映射：先看 `kargs`（按名），再看 `positionals`（按位置），都不中则视为默认省略。设计思想：把"实参与形参的对应关系"不做黑盒猜测，而是由语言规范与内省 API 共同推出——这类"基于契约的推理"是自己写框架时的核心素养。

39.39 Open Issues (遗留问题)

英文原文

Although our range-testing tool works as planned, there are three caveats remain—it doesn't detect invalid calls, doesn't handle some arbitrary-argument signatures, and

d doesn't fully support nesting. Invalid calls: Calls that are not valid still fail in our final decorator—e.g., `omitargs()` and `omitargs(d=8, c=7, b=6)` both raise `TypeError` for a missing positional argument `a`. These fail though only when we actually call the original function at the end of the wrapper; since the call would fail anyway, we might as well let Python's own argument-matching logic detect the problem. Arbitrary arguments: Although handled for the ordinary modes, the final version doesn't handle `pargs` and `kargs` names used as the decorated function's own arguments. If an extra keyword is passed, its name will show up in `kargs` and can be tested normally; if not passed, it's treated as though defaulted; if an extra positional is passed, there's no way to reference it—its name won't be in either collection. In other words, it tests arbitrary keywords by name but not unnamed positionals beyond the signature. Decorator nesting: The approach doesn't fully support decorator nesting in which multiple stages are combined. Because it analyzes arguments using names in function definitions, proxies change argument-name signatures at their levels; as coded, only the most-deeply-nested validations run in full. (Position-based solution supports proxies better but not keywords.)

中文翻译

虽然我们的区间测试工具按计划工作，但仍留三个警告——它不检测无效调用、不处理某些任意参数签名、也不完全支持嵌套。无效调用 (invalid calls)：对原函数合法的"调用"仍然会在包装器里失败——例如 `omitargs()` 和 `omitargs(d=8, c=7, b=6)` 都会因缺少位置参数 `a` 而抛 `TypeError`。但这只在包装器末尾真正调用原函数时才失败。反正这个调用本就要失败，不如就让 Python 自己的参数匹配逻辑替我们抓问题。任意参数 (arbitrary arguments)：虽然普通模式都已处理，最终版还是没有显式处理"被装饰函数自身使用 `pargs/kargs` (星号收集器)"的情况。若多传了关键字 → 名字会出现在 `kargs`，可按名正常测试；若没传 → 视同默认而被跳过（虽然它是可选额外参数）；若多传了位置参数 → 根本没法引用它（不在期望参数对照里，也无法映射相对位置）。即：按名的任意关键字天然支持，但超出签名的"无名位置参数"无法挂载校验。关于调用里的星号展开（如 `f([1], 2, [3]...)`）——它们在到达装饰器之前就被展开 (flattened)，对装饰器代码毫无影响，所以可以安然无忧。装饰器嵌套 (decorator nesting)：此方案并不完全支持"用嵌套把多级步骤组合起来"。因为它用函数定义里的名字分析参数，而嵌套装饰返回的代理 `onCall` 的调用签名与"原函数或装饰器参数中的参数名"并不对应——当嵌套时，只有最内层出现的校验会完整运行；其余层只对"按名传入"的参数抛校验（按位置传的会被当作省略默认值而跳过）。这正是"位置派" (`rangetest0`) 与"名字派" (`rangetest`) 各残余半边的体现：位置派更支持代理但不懂关键字，名字派反过来。

深度理解

- 核心概念：再好的工具也有"假设边界"——本章诚实地罗列三条：非法调用、任意参数签名、嵌套场景。
- 底层实现：嵌套失效的根因——onCall 代理的函数签名是 (pargs, kargs)，它"期望参数名元组"在装饰时刻被换成别的名（如 onCall 自己的 argnames），外层的名字映射就错位了。
- 设计原因：作者故意不把工具做到"全能"——宁可"明确边界"也不"复杂到不可维护"，并把扩展列为练习。
- 解决问题：提醒我们"通用工具"的谦虚心态——知道自己的失效模式，好过假装全能。
- 初学者误区：想当然认为"装饰器嵌套"就该像括号一样天然工作——嵌套推理与参数名映射是最容易想当然的位置。

代码分析

```
#
>>> def f(a, b, c, d, e, f, g, h, i): pass
>>> f(*[1], 2, *[3], 4, f=6, *[5], **dict(g=7), h=8, **{'i': 9})
# (1, 2, 3, 4, 5) {'f': 6, 'g': 7, 'h': 8, 'i': 9}
```

执行链：Python 对调用的展开发生在"进入函数体之前"，故装饰器里的 pargs/kargs 只看到"已扁平化"的值。设计思想：理解"调用压缩先于被调用者"，就能避免在装饰器里重复实现展开逻辑。

39.40 Decorator Arguments Versus Function Annotations (装饰器参数对比函数注解)

英文原文

In closing, Python's annotation feature (introduced in Chapter 19) could also provide an alternative to decorator arguments for specifying the valid ranges in our example. We could use annotations to code range limits in the header line instead of in decorator arguments. We would still need a decorator, but we would essentially trade decorator-argument syntax: `@range test(a=(1, 5), c=(0.0, 1.0)) / def func(a, b, c): ...` for annotation syntax like this: `@rangetest / def func(a:(1, 5), b, c:(0.0, 1.0)): ...` That is, the range constraints would be moved into the function itself instead of being coded externally in a decorator line. Example 39-27 illustrates the structure of the resulting decorators under both schemes. The annotation version requires one fewer level of nesting because it doesn't need to retain decorator arguments as state. In fact, using annotations instead of decorator arguments in this example actually limits its utility. By moving the validation specs into the `def` header, we essentially commit the function to single role—annotation directly allows only one expression per argument, so it can have only one purpose. By contrast, because decorator arguments are coded outside the function itself, they are easier to

o remove and more general. Crucially, by nesting decorators with arguments, we can often apply multiple augmentation steps to the same function. Still, if you have a single purpose in mind, the choice is largely stylistic and subjective. As is so often true in life, one person's decoration may well be another's syntactic clutter.

中文翻译

最后提一句：Python 的注解（annotation）特性（第 19 章引入）也能为“指定测试区间”提供装饰器参数的替代方案。我们可以用注解把区间限制写进 def 头行，而不是写进装饰器参数。我们仍然需要一个装饰器去包装函数、拦截调用，但本质上是用“装饰器参数”语法换“注解”语法：`@rangetest(a=(1, 5), c=(0.0, 1.0)) / def func(a, b, c): ...` → 换为 `@rangetest / def func(a:(1, 5), b, c:(0.0, 1.0)): ...` 也就是说，区间约束从“装饰器行的外部配置”，移到了函数内部。示例 39-27 给出了两种方案下结果装饰器的结构。注解版因为不必保留装饰器参数作为状态，少了一层嵌套。两种方案运行输出一致：都打印 `{'a': (1,5), 'c': (0.0,1.0)}`。不过实际上，这个工具上用注解替代装饰器参数会限制用途。把校验规范挪进 def 头，相当于给函数“赋予单一角色”：注解规定每个参数只能有一个表达式，于是参数只能有一种用途（比如区间校验）——无法再兼作类型提示（第 6 章）。而装饰器参数在函数之外，既更容易去掉，又更通用；关键还有，通过嵌套带参装饰器，我们能对同一函数施加多个增强步骤。从审美与主观角度，两者之争基本是风格之争。”如生活常事：一个人的装饰，可能是另一个人的语法堆砌。

深度理解

·核心概念：同一桩"规范校验配置"，两个载体——外部（装饰器参数）vs 内部（注解）。二选一其实是"职责是否移出函数"的选择。

·底层实现：注解版直接 `func.annotations` 取字典；少一层闭包，因为无需先接参数。新代码骨架：
`def rangetest(func):
 def onCall(pargs,kargs):
 argchecks = func.annotations ...`

·设计原因：注解"每参数一个表达式"的固有局限——无法多用途复用；装饰参数则天然支持"多装饰器叠加多个角色"。

·实际选择：类型提示最好用它自己的机制（typing）；运行期校验用装饰器参数更清晰。

·初学者误区：以为注解就是"类型声明"。注解纯粹是写入 `annotations` 字典的表达式，解释器不主动校验——可当元数据、可运行时读取。

代码分析

```
#
def rangetest(**argchecks):
    def onDecorator(func):
        def onCall(*pargs, **kargs):
            print(argchecks)          # {'a': (1,5), 'c': (0.0,1.0...
            for check in argchecks: ... #
            return func(*pargs, **kargs)
        return onCall
    return onDecorator

#
def rangetest(func):
    def onCall(*pargs, **kargs):
        argchecks = func.__annotations__ #
        print(argchecks)                 # {'a': (1,5), 'c': (0...
        for check in argchecks: ...      #
        return func(*pargs, **kargs)
    return onCall
```

执行链：注解版在 @rangetest 时 func 已编译、func.annotations 已就位，包装器每次调用动态读取。设计思想：同一个“区间表”，存“闭包外层”还是存“函数属性”二者等价、界面不同——这正是“状态容器选择决定 API 姿态”的又一印证。

39.41 Chapter Summary (本章小结)

英文原文

In this chapter, we explored decorators—both the function and class varieties. As we learned, decorators are a way to insert code to be run automatically when a function or class is defined. When a decorator is

used, Python rebinds a function or class name to the callable object that the decorator returns. This hook allows us to manage functions and classes themselves or later calls to them. By adding a layer of wrapper logic to catch later calls, we can augment both function calls and instance interfaces. Decorators provide an explicit and uniform way to achieve such goals. As we also learned, class decorators can be used to manage classes themselves rather than just their instances. Because this functionality overlaps with metaclasses—the topic of the next and final technical chapter—you'll have to read on for the conclusion to this story and that of this book at large. First, though, let's work through the following quiz. Because this chapter was mostly focused on its examples, the quiz will ask you to modify some of its examples' code in order to review their concepts. Both the examples' code and the quiz's solutions are located in the book's examples package (see the Preface for access pointers). Look for the solutions' code there in this chapter's QuizAnswers subfolder. If you're pressed for time, you're welcome to jump right into studying the solutions; programming is often as much about reading code as writing it.

中文翻译

在本章中，我们探索了装饰器——函数装饰器与类装饰器两大门类。如我们所学，装饰器是一种“在函数或类被定义

时插入自动运行代码"的方式。使用装饰器时，Python 会把函数或类名字重绑定到装饰器返回的可调用对象上。这让我们能管理函数与类本身，或管理对它们的后续调用。通过加一层包装代码来拦截后续调用，我们既能增强函数调用，也能增强实例接口。装饰器为达成这些目标提供了显式而统一的方式。我们还学到，类装饰器不仅能管理实例，也能管理类本身。由于这个功能与元类（metaclasses）——下一章也是最后一个技术章节的主题——重叠，要想听到这个故事和本书整体大故事的结局，你得继续读下去。不过先别急，让我们做做下面的测验。因为本章重心全在示例上，测验会让你通过修改某些示例代码来复习相关概念。示例代码与测验解答都在本书的 examples 包中（请参照前言获取入口）。解答代码在这章的 QuizAnswers 子文件夹里。如果时间紧张，欢迎直接去研读解答——编程往往一半是读代码，一半是写代码。

深度理解

- 核心概念：收束全章的三个关键词——自动代码插入、名字重绑定、包装层。
- 底层实现：全章所有工具（tracer、timer、singleton、Tracer、Private/Public、rangetest）都是"重绑定 + 代理 + 状态容器"三个基本件的不同组合。
- 设计原因：作者把"类装饰器 vs 元类"的接力棒交给下一章——类装饰器是元类的"轻量替身"，理解前者是理解后者的引子。
- 解决问题：本章工具都可直接复用（计时器、区间检查器），是"书里写给你的可搬运工具"。

·初学者误区：以为"读懂了 @ 语法"等于"学会了装饰器"。真正要带走的是对"状态在何处留存"函数与方法如何兼容"内置运算如何接"三条脊梁的理解。

39.42 Test Your Knowledge: Quiz (测验)

英文原文

1. Method decorators: As mentioned in one of this chapter's notes, the `timerdeco2.py` module's call-timer decorator (Example 39-10, "Adding Decorator Arguments") works only on plain functions because it uses a nested class with a call to intercept calls. Rewrite it so that it works on both plain functions and methods in classes. 2. Class decorators: Add a debug check to the `Private/Public` class decorators in the `access2.py` module (Example 39-18), so that when Python is run with `-O`, no wrapping is done—and thus no performance costs. Also, use a mix-in superclass to catch a few built-in operations. 3. Generalized argument validation: Generalize the range test decorator in `rangetest.py` (Example 39-25) so it can be used for multiple kinds of argument validations (e.g., type testing) in a single decorator. Passed-in validation functions may be the simplest solution.

中文翻译

1. 方法装饰器 (method decorators) : 本章一处提过, `timerdeco2.py` 模块里的调用计时装饰器 (示例 39-10, "添加装饰器参数") 因使用"带 `call` 的嵌套类"来拦截调用, 只能用于普通函数。请重写它, 让它能同时用于简单的函数与类中的方法。(提示: 参考"类陷阱: 装饰方法", 很可能需要函数对象的属性来保存总耗时, 因为你将不再有嵌套类可留存状态, 也无法从装饰器代码外部访问 `nonlocal`。) 2. 类装饰器: 给 `access2.py` (示例 39-18) 中的 `Public/Private` 类装饰器加 `debug` 检查, 使 Python 以 `-O` 运行时不做任何包装——从而消除性能开销。另外, 用本章学过的混入超类技术捕获几个内置运算。3. 通用参数校验: 把 `rangetest.py` (示例 39-25) 中的区间测试装饰器泛化, 使同一套代码能做多种参数校验 (如类型测试等)。传入校验函数可能是最简方案 (子类提供期望方法也常能提供类似的泛化途径)。这题相当有挑战性, 务必参考解答。

39.43 Test Your Knowledge: Answers (测试答案)

英文原文

As noted, coding solutions for this quiz are in this chapter's `QuizAnswers` subfolder of the book examples package. Each of the three problems is documented there, along with a plain-text notes file. Because answers are essential to reviewing the concepts, and this section intentionally leaves room for self-study, we'll point you there rather than reproducing the answers i

nline.

中文翻译

如前所述，本测验的编码解答位于本书示例包的本章 Quiz Answers 子文件夹中。每个问题都附有更详细背景说明（Notes.txt 纯文本文件）。本版教材选择把这些解答放到网上而不是这里列出——因为这样一来能省大约 10 页纸，也因为“互联网这玩意说不定真的会火起来”。

深度理解

- 重写计时器（题 1）：核心是把嵌套类 Timer 换成嵌套函数 wrapper，用 wrapper.calls/alltime 保存累计状态——一句话：放弃“可调用类”，改投“嵌套函数 + 函数属性”。这正好复习了“方法陷阱”与“状态容器”两条主线。
- debug 提速（题 2）：if not debug: return aClass（即不改包装），实现“-O 即免装饰”。它是 rangetest 已演示开关策略对类装饰器的移植。
- 泛化校验（题 3）：把“范围判断”抽象成回调函数 validate(arg)，装饰器只做“路由 + 转发”——单一骨架、多态规则。
- 方法论：三题共同训练同一复盘——状态容器选择、debug 开关、规则回调化，正是本章案例的三个精髓。

本章总结

技术拓展 (Technical Expansion)

- 实际项目中的应用场景
 - Flask/FastAPI 的路由注册 (`@app.route('/path')` 本质就是"装饰器参数 + 引用注册")、Django 的 `@loginrequired` (函数/方法鉴权包装)、PyTorch 的 `@torch.no_grad()`、pytest 的 `@pytest.fixture`。
 - `functools.lru_cache` 是"带参数的装饰器工厂", 与本章 `@timer(label=...)` 结构完全同构——先接受配置、再接受函数。
 - 你自己写 Web 框架时, `@app.route`、`@middleware`、参数校验 `@validateschema(...)` 都是本章三体术 (路由/开关/回调) 的直接落场。
 - 与其他语言 (Java/C++) 的区别

语言	类似机制	本质	Python 差异
Python	装饰器	运行时名字重绑定	调用封装、无文本展开
Java	注解 (annotation)	编译期元数据	注解默认不改行为, Python 装饰器真改行为
C++	宏 #define	预处理文本替换	与 import vs #include 同级的
Rust	过程宏	编译期代码生成	属"文本/语法层", 非运行期对象操作

Python 装饰器与这些"代码生成"路线最根本的区别: 它是运行时的对象操作, 一切皆可推迟到 import/调用时刻, 灵活但少了编译期保证。

· Python 发展历史背景：装饰器的语法灵感可追溯到 Python 2.4 引入的"函数装饰器"，其语法形式整体借鉴自 Java 注解；类装饰器与参数式装饰器在 Python 3.0 面世；此后 `functools.wraps`（保留 `name/doc` 元数据）、`wrapped` 等配套标准被补齐。Python 3.9 才允许 `@` 后跟任意表达式；3.12 又允许对 `@` 表达式里的复合 f-string 等更自由的写法。作者在书里标注了大量 "Python 3.12" 实例（如嵌套 f-string 的 `trace` 打印）。

·高级开发者需要掌握的相关知识

主题	作用
<code>functools.wraps</code>	把被包装函数的元数据（ <code>name</code> 等）复制到 <code>wrapper</code> ，保证测试/文档工具正常
<code>functools.cachedproperty</code> / <code>lru_cache</code>	官方缓存装饰器，理解其"多级状态"设计
参数内省 <code>inspect.signature</code>	现代替代 <code>code.covarnames</code> 的规范做法
元类（第 40 章）	装饰器+元类的职责边界与配合
类型注解 + 装饰	<code>typing</code> 与 <code>@</code> 组合出契约式校验框架

学习建议 (Learning Advice)

·重要程度：5/5 星（必读）。装饰器与闭包、描述符并称 Python"高级三件套"，是通往框架源码、元编程的必经关。

·应该掌握到什么程度：

1. 能把"任意 `@dec`"翻译成"`名字 = dec(名字)`"并反义。
2. 能写出 4 种状态载体（实例属性/global/nonlocal/函数属性）版本的 `tracer`，并说清差异。

3. 能指出"类包装器 + call"为何在方法上翻车，改用嵌套 def。
4. 理解两级类装饰器（拦截实例创建 vs 拦截属性访问）与两级代理陷阱。
5. 能读懂 getattr / setattr / get 三者合成的大案例（access2、rangetest）。

·后续学习方向：

1. 第 40 章「元类（metaclasses）」——理解"类装饰器 = 元类的轻量版"这一对偶关系。
2. 重新阅读第 31 章委托模式、第 38 章描述符与 property，把"钩子机制"串成一张网。
3. 动手改这三道章末测验题（计时器方法化、debug 免包装、校验回调泛化），并对照 QuizAnswers 子文件夹的官方解答。
4. 用 functools.wraps、inspect.signature、singledispatch 重写本章工具，体会标准库对它们的使用。

全章完。装饰器是 Python 的"钩子文化"最集中的体现——从 @staticmethod 到最终大的属性隐私与参数校验，全部是"定义后插入代码"这一动作的不同姿态。把本章代码亲手敲一遍、跑一遍、改一遍，比读懂任何一段总结都更重要。